更多学习资料，请关注淘宝店铺：学神资料站　https://shop156050543.taobao.com/

What to Feed a Gerrymander

Control # 1421

Mathematics Contest in Modeling
February 12, 2007

Abstract

Gerrymandering, the practice of dividing political districts into winding and unfair geometries, has a deleterious effect on democratic accountability and participation. Incumbent politicians have an incentive to create districts to their advantage (California in 2000, Texas in 2003) so one proposed remedy for gerrymandering is to adopt an objective, possibly computerized, methodology for districting. We present two computationally efficient algorithms for solving the districting problem by modeling it as a Markov decision process rewarding traditional measures of district "goodness": equality of population, continuity, preservation of county lines, and compactness of shape. Our *Multi-Seeded Growth Model* simulates the creation of a fixed number of districts for an arbitrary geography by "planting seeds" for districts and specifying particular growth rules. The result of this process is refined immensely in our *Partition Optimization Model* which uses stochastic domain hill-climbing to make small changes in district lines that improve goodness. We include, as an extension, an optimization to minimize projected inequality in district populations between redistrictings. As a case study, we implement our models to create an unbiased, geographically simple districting of New York using tract-level data from the 2000 Census. We conclude with an open letter to members of the New York State Assembly.

What to Feed a Gerrymander

Team 1421

1 What is Gerrymandering?

Gerrymandering is the division of an area into political districts that give special advantages to one group. Frequently, this involves concentrating "unfavorable" voters in a few districts to ensure that "favorable" voters will win in many more districts. In order to squeeze all of the unfavorable voters into a few districts, gerrymandering creates snaky and odd shaped regions. The eponymous label was created when politician Elbridge Gerry pioneered this technique in early 19th Century and his opponents claimed the districts resembled salamanders.

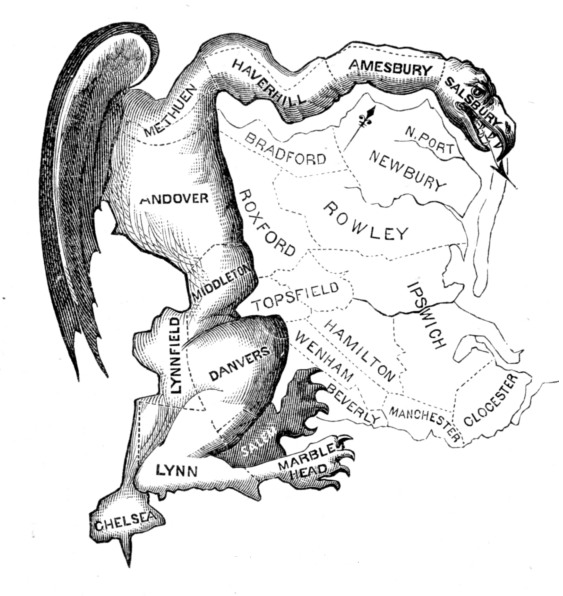


Figure 1: The original "Gerry-mander" from the *Boston Centinel* (1812)

1.1 Basic Terminology

- **Packing** - Forcing a disproportionately high concentration of a particular group into one district to lessen their impact in nearby districts.
- **Cracking** - Spreading out members of some group in several districts in order to reduce their impact in each of these districts.
- **Forfeit district** - A district where group A packs the members of group B so that group B wins this district but loses several surrounding districts which B may have won with a different districting scheme.

- **Wasted Vote** - A vote cast by a member of group A in a district where A is already assured victory so voting has no bearing on the result. In general, the group with more wasted votes is made worse off by a districting plan.

1.2 Why is it so bad?

Politicians today still gerrymander federal and state-level electoral districts and the public outcry is still strongly negative. Before we set out to eliminate this practice we should discuss why gerrymandering is considered problematic.

First off, gerrymandering reduces electoral competition within districts since cracking/-packing makes elections uncompetitive. Further, incumbent representatives are in no real danger of losing elections so they do not campaign vigorously which can lead to lower voter turnout. Exacerbating the problem, incumbents' increased advantage means they are less incentivized to govern based on their constituents' interests so democratic accountability and engagement mutually deteriorate.

Gerrymandering also presents the practical problem that it is difficult to explain to voters why district shapes are so labyrinthine. Some districts connect demographically similar but geographically distant regions using thin filaments such as the district depicted in Figure 2. "Niceness" of district shape almost always takes a back seat to political and racial concerns when districts are being created. Example: In the 2000 California realignment, Democrats and Republicans united to design incumbent-favoring districts which resulted in the reelection of all of the 153 involved legislators in 2004. How can one argue that this is in voters' best interests?

However, it should be noted that gerrymandering can be considered appropriate in specific situations. For instance, the Arizona Legislature gerrymandered a division between the historically hostile Hopi and Navajo tribes even though the Hopi reservation is entirely surrounded by the Navajo reservation.

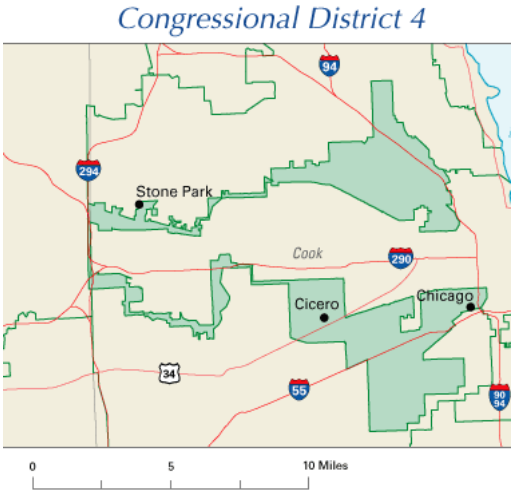


Figure 2: A present-day gerrymander, the Illinois 4th congressional district. (The two "earmuffs" are connected by a narrow band along Highway 294.)

1.3 The legality of gerrymandering

We should be clear on one point: though gerrymandering is objectionable to many, it is legal around the country. Interestingly, the Voting Rights Act of 1965 which eliminated poll taxes and other discriminatory voting policies may have inadvertently increased the prevalence of gerrymandering. One interpretation of the Act was that it mandated nondiscriminatory election *results* which led to a strange reversal of vocabulary where creating "majority-minority" districts was considered beneficial. These gerrymandered districts were packed with minorities which guaranteed minority representation in Congress.

However, in Shaw v. Reno (1993), and later in Miller v. Johnson (1995), the Supreme Court ruled that racial/ethnic gerrymanders were unconstitutional. Nevertheless, Hunt v. Cromatrie approved of a seemingly racial gerrymandering since the motivation was mostly partisan rather than racial. The recent case League of United Latin American Citizens v. Perry (June 2006) upheld the position that states are free to redistrict as often as they like so long as these redistrictings follow are not purely racially motivated.

2 Assumptions and Notation

2.1 What can we consider when districting?

We have compiled the following list of possible factors one might consider is districting a State. The list is ranked with factors we consider more important or legitimate at the top.

1. Population equality between districts (legally mandated)
2. Continuity of districts (legally mandated, excepting islands)
3. Respect for legal boundaries (counties, city limits, townships)
4. Respect for natural geographic boundaries
5. Compactness of district shapes
6. Respect for man-made boundaries (highways, parks, etc.)
7. Respect for socio-economic similarity of constituents
8. Similarity to past district boundaries
9. Partisan political concerns
10. Desire to make districts (un)competitive
11. Racial/ethnic concerns
12. Desire to protect (or unseat) incumbent politicians

We consider only the top seven factors in our model. Factors 9-12 are all related to political or racial concerns which our model is specifically designed to ignore. The case SC State Conference of Branches v. Riley (1982) ruled that past districts (Factor 8) are a legitimate tool for creating new districts but we choose to ignore past districts since they are heavily biased by Factors 9-12.

2.2 Geography and similar characteristics

The US Census Bureau provides a great deal of data on legal, natural, and man-made boundaries as well as socio-economic similarity of regions. In each census, the United States is broken up into several degrees of accuracy, the smallest of which are: blocks (40 people on average), block groups (1500 people), and tracts (4500 people).

We follow the practice in Young (1988) by districting based on a maximum level of resolution which in our Case Study (Section 5) is census tracts. Notational note: we refer to the smallest unit of division generally as a *tract*.

A reference from the Caliper Corporation describes tracts in the following quotation:

> Census tract boundaries normally follow visible features, but may follow governmental unit boundaries and other non-visible features, and they always nest within counties. Census tracts are designed to be relatively homogenous units with respect to population characteristics, economic status, and living conditions at the time the users established them.

For these reasons we believe that units at the tracts size (or less) are acceptably small and homogenous to use as a base unit. Further, tracts are completely contained within counties so we can easily check whether or not a district breaks county integrity.

2.3 Notation

Define m to be the number of census tracts, and n the number of districts.

We denote our districts by $D_i, 1 \leq j \leq n$, and our tracts by $T_l, 1 \leq l \leq m$. Denote the set of all tracts by $\Gamma = \{T_l\}_{1 \leq l \leq m}$; we call this a *State*. Denote the set of all districts at a particular time by $\Delta = \{D_i\}_{1 \leq j \leq n}$. We call this a *partition* for the State.

2.3.1 Adjacency

Define the symmetric relation $T_p \sim T_q$ for tract pairs (T_p, T_q) which are adjacent. Define the function $d(T_l)$ to be the district to which the tract T_l belongs. We also naturally extend the definition of d to sets of tracts.

Define the *neighbor set* of tract T_l by $a_T(T_l) = \{T_p \in \Gamma | T_l \sim T_p\}$ to be the set of all census tracts neighboring T_l, and define $a_D(T_l) = d(a_T(T_l))$ to be the set of all districts containing neighbors of T_l. Every tract borders at least one other tracts, so $a_T(T_l)$ and $a_D(T_l)$ have cardinality at least one for all T_l.

2.3.2 Borders

Define the *border* of district D_i as $\partial D_i = \{T_l \in D_i | a_D(T_l) \neq \{D_i\}\}$ which is the set tracts in D_i that are adjacent to at least one district other than D_i. The *interior* of district D_i is $I_i = D_i \backslash \partial D_i$, the set of census tracts in D_i whose neighbors are all in D_i. Denote the total number of tracts in district D_i as $m_i = |D_i|$ the number of border tracts as $b_i = |\partial D_i|$.

The *frontier* of D_i is denoted $F_i = (\cup_{T_l \in D_i} a_T(T_l)) \backslash D_i$, i.e. the set of all tracts outside of D_i that border the boundry tracts of D_i.

2.3.3 Counties

We denote a county as C_j and the set of all counties as Λ. Districts can (and often do) break county boundaries but tracts are contained entirely within counties so we can think of a county as a set of districts. Districts are also sets of tracts so we interpret the set intersection $D_i \cap C_j$ as the set of tracts in both district D_i and county C_j. From this, we define $c(D_i) = \{C_j | D_i \cap C_j \neq \emptyset\}$ to be the set of counties which overlap with D_i.

2.3.4 Population

Let the population of our State be P and we denote the optimal district size, $\frac{P}{n}$, as $\bar{p}$. We use the function $p(\cdot)$ to generally denote the population of an object, for instance $p(T_l)$ and $p(C_j)$ are the populations of tract T_l and county C_j, respectively. Due to frequent use, we use the shorthand $p_i = p(D_i)$ for the population of districts.

Table 1 is a useful reference of these numerous definitions.

Table 1: Variables and their meanings

Variable	Definition
n	Number of congressional districts
D_i	The ith district ($1 \leq i \leq n$)
Δ	Set of all districts in a State, a *partition*
m	Number of census tracts
T_l	The lth tractfin ($1 \leq l \leq m$)
Γ	Set of all tracts in a State
$d(T_l)$	District to which tract T_l belongs
$T_p \sim T_q$	Tracts T_p and T_q are adjacent
$a_T(T_l)$	Set of tracts adjacent to tract T_l
$a_D(T_l)$	Set of districts containing tracts neighboring T_l
∂D_i	Border of D_i, tracts that neighbor another district
I_i	Interior of D_i, tracts with do not neighbor another district
m_i	Number of tracts in D_i
b_i	Number of tracts in ∂D_i
F_i	Set of all tracts outside of D_i that border ∂D_i
C_j	The jth county
$c(T_l)$	The county to which tract T_l belongs
$c(D_i)$	The set of counties containing district D_i
P	Total population of the State
$\bar{p}$	Average population of a district
$p(\cdot)$	Population of an arbitrary object
p_i	Shorthand for $p(D_i)$, population of district D_i

2.4 Past Models

Prior to explaining our modeling approach we would like discuss some previous work in the literature on congressional districting and gerrymandering. We used these papers as guides as we thought about and further refined our algorithm and implementation.

Cirincione *et al.* (2000) judge the quality of a districting plan based on equal population, preservation of county integrity, and district area compactness. They require that district populations differ by no more than 1% from exact equality in the number of constituents, and require point contiguity of the districts. The algorithm constructs districts by picking a random block group (their unit size), then adding additional block groups to the new district until the district population reaches $\bar{p}$. At this point they repeat the process starting with a new random block group. Compactness of districts is based on their minimum bounding rectangles and county integrety is encouraged by "randomly" selecting new block groups with a preference for block groups in already inhabited counties.

Mehrotra *et al.* (1998) and Garfinkel and Nemhauser (1970) implement a "branch-and-price" method in the optimization step. They first obtain a districting, and optimize over their constraints such that population values are allowed to vary in the final solution of the optimization step. In a final step they split up population units to ensure population equality. They define compactness in a graph-theoretical manner where connected nodes are adjacent tracts. They define the "center" of a district to be the node (tract) with the lowest maximum distance to another other tract. They consider a graph (district) more compact when sum of distances from each node to the center is small.

We do not use this measure, as it does not uniquely define the center of a graph, and, contrary to their claims, does allow for oddly-shaped districts, such as a district whose graph is a star-shaped tree with one tract in the center and many non-contiguous paths emanating from it. In our case study simulations, prior to the incorporation of a compactness factor in the objective function, we often obtain such a tree structure, which is one of the salient features of gerrymandering.

We also do not use a "branch-and-price" method of optimization. Following suggestions of Nagel (1965) and Kaiser (1966), we employ a local search algorithm in which tracts are swapped between existing districts to maximize some objective function. We describe this process in detail in Section 4.

2.5 Measuring compactness

The notion of compactness of a planar region has no uniformly accepted definition and research done by Young (1988) suggests that any reasonable measure of compactness fails to work well for certain geographic configurations. He further suggests that any good measure of compactness in such problems should consider the population units (census tracts in our case study) as indivisible units, and therefore that the measure of compactness should be made independently of the predetermined shapes of the population units. We follow this directive in our definition of compactness.

In fact, the compactness measures given in Young (1998) are not reasonable in the first instance, and do not include any notion of the area of a district, or comparing it to the perimeter. The measures include the maximum total perimeter of a district in a districting, determining the relative height and width of the district, and finding the moment of inertia

of the district. All of these measures fail to consider both perimeter and area simultaneously, which seems to be a reasonable requirement of a good compactness measure.

The Isoperimetric Theorem, first proved (non-rigorously) by J. Steiner in 1838, states that the quantity A/P^2, given by the ratio of the area A of a planar region (not necessarily continuous) to the square of its perimeter is maximized when the region is circular. The maximum achievable compactness, that of a circle with radius r, is given by $\frac{\pi r^2}{4\pi^2 r^2} = \frac{1}{4\pi}$ so we define *compactness* of a region as the ratio $(4\pi A)/P^2$. This ratio is bounded within $(0, 1]$ where higher values indicate greater compactness.

We believe this serves as a good measure of the broadly defined "regularity" of a region which is so important to the study of Congressional districting and gerrymandering. Specifically, any shear of factor s applied to a circle decreases the compactness by a factor of s, and any concave region has lower compactness than does its convex hull. It is easy to see that we can make an even stronger statement: the convex hull of a concave region has greater area *and* smaller perimeter.

Observe that a square gets close to the optimum, with a compactness of $\frac{4\pi}{16} \approx 0.785$. This implies that the set of possible compactness values for rectangles is $(0, 0.785)$ since a square is the most compact rectangle.

3 The *Multi-Seeded Growth Model*

We take a two-stage approach to finding the best districts for a given State. In the *Multi-Seeded Growth Model*, referred to as *MSGM* hereafter, we find an initial allocation of n districts so that the partition has modest levels of population equality and county preservation. Our more precise *Partition Optimization Model*, or *POM*, edits and improves the rough sketch from *MSGM* into until it becomes, hopefully, a work of art.

The reason that our model runs in two phases is simple: speed. Our knee-jerk reaction to the problem was to randomly allocate tracts to the n districts and then optimize by swapping tracts trying to improve some objective function. However, a random initial configuration is so far from the global maximum that the search might take millions of years.

The *MSGM* generates a very crude coloring of a State that ensures district contiguity and tries, but does not guarantee, to achieve population equality and county preservation. The districts created by *MSGM* are completely unacceptable for an actual plan but save enormous amounts of computing time for our solution.

3.1 How it works

At first, our task seems daunting. How do we allocate n districts equally, even to a rough approximation? Our solution is to grow the n districts simultaneously until they cover the State.

We start by allocating the entire State to a blank, dummy district D_0, and then allocating n tracts that serve as the initial "seeds" for the final districts, such that each $D_i, i \in \{1, \ldots, n\}$ begins as only a single tract. Now while $|D_0| > 0$, we take the set S of all

possible moves which involve taking a district from D_0 while preserving contiguity. That is:

$$S = \bigcup_{i=1}^{n} \bigcup_{T_l \in F_i} M(T_l, D_0, D_i)$$

Where $M(T_l, D_i, D_j)$ represents a *move of tract* T_l *from* D_i *to* D_j, corresponding to the exit of T_l from D_i and the entrance of T_l into D_j. We then sort the moves in S by our heuristic function $\Psi(D_1, \ldots, D_n) \to \mathbb{R}$, a function increasing in the desirability of our prospective partition. Each move is scored by the heuristic value that would result if we were to accept only that move. We then conclude by performing the moves corresponding to the top 3% of the scored moves in S. Note that this method preserves contiguity, because by definition any $T_l \in F_i$ must be contiguous with D_i, and thus the D_i are contiguous at each step.

Had we but world enough, and time, we would only perform the best possible move found in S before recalculating the frontier. Even though in the *MSGM* we do not consider moves between two "true" districts (rather, we consider only moves between a true district and the dummy district), the value of a move does not exist in isolation. Consider two distinct districts D_i and D_j, and two tracts $T_l \in F_i \cap F_j$ and $T_k \in F_i \cap F_j^c$. The acceptance of $M(T_k, D_0, D_i)$ alters the heuristic value of every move associated with F_i, which could potentially affect the optimality of further moves with D_i, such as the acceptance of $M(T_l, D_0, D_i)$ rather than $M(T_l, D_0, D_j)$. Furthermore, the acceptance of $M(T_l, D_0, D_i)$ likely expands the size of F_i. Perhaps there is an optimal move opened up in this new frontier that we do not even consider, because we have not even calculated its value.

It would be better to only perform the best move, but such a strategy was found to be too computationally intensive. We compromise by taking only a small, elite fraction of the moves in each step before recalculating S and the values of its associated moves. In this respect, our approach is analogous to the strategy of *modified policy iteration* for solving a Markov decision problem. And just as modified policy iteration excels in practice, we found that the tradeoff of possible inefficiency is more than compensated for by the speed gains of the algorithm, especially considering that the solution obtained by *MSGM* will be further refined by *POM*.

In true modified policy iteration, k rounds of value iteration are made in-between policy iterations, such that k is fixed. Our *MSGM* scheme uses a variable number of moves in-between recalculating the value of the frontier. We selected our scheme because it causes us to be delicate in our selections of tract allocations, making moves virtually one-at-a-time, at the beginning and end of the *MSGM*. By focusing on the beginning and end of the problem, we attempt to avoid having a single district grow too large through possible inefficient allocation.

Unlike Cirincione (2000) we use random initial seeds weighted by population rather than seeds that are equally spaced around the State. The process works as follows: while there are still random seeds to be selected, we find a candidate initial seed tract T_l in D_0. Letting the largest tract in our State have population $\hat{p}$, we accept T_l as an initial seed with probability $p(T_l)/\hat{p}$. We thus select tracts in linear proportion to their population. We found that the *MSGM* algorithm produces the best initial results when all the districts have the same amount of *population*, rather than the same *number of tracts* around which to grow. The geographically optimal placement of five, or fewer, starting seeds in the NYC Metropolitan

area and Long Island evinces the fallibility of the equidistant initial seed method.

We have presented our scheme for growing emerging districts, but we should also discuss the heuristic by which we rank candidate moves. It has two components: a population score and a county score.

3.2 Population score

Even thought the *MSGM* is only an rough start for our optimization we would like to minimize egregious disparities in population between districts. We would much prefer if the *MSGM* produces a result where the largest district has twice the population of the smallest rather than 100 times the population.

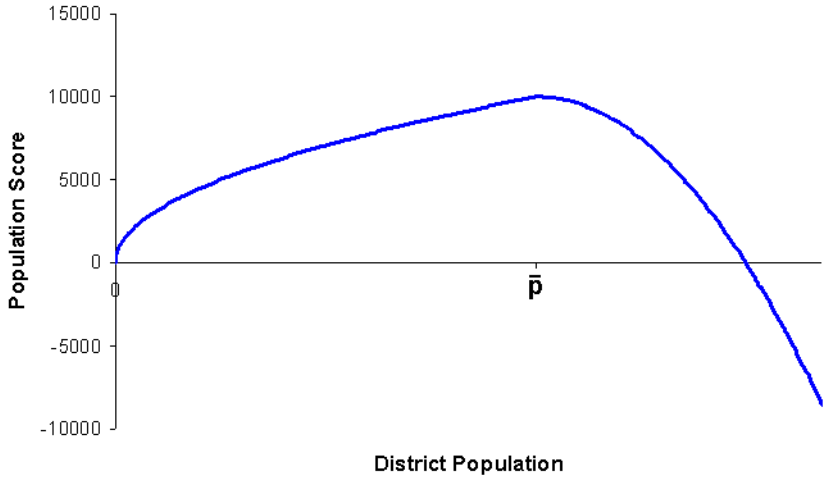
Clearly, the population component of our heuristic should give the highest score to a district when $p_i = \bar{p}$. Additionally we want to penalize large deviations from the optimal population level so our function should be concave down.

Admittedly, choosing a heuristic is somewhat arbitrary but this does not bother us since the results from *MSGM* are only a baseline. Let $f(p_i)$ be the population heuristic score for a district with population D_i. We use a piecewise definition for f:

$$f(p_i) = \begin{cases} M\sqrt{\frac{p_i}{\bar{p}}}, & \text{if} p_i \leq \bar{p} \\ M - \frac{4M}{p_i^2}(p_i - \bar{p})^2, & \text{if} p_i > \bar{p} \end{cases} \tag{1}$$

Notice that f is steeper for values $p_i > \bar{p}$ because we do not want *growing* districts to engulf too much population; we penalize deviations above $\bar{p}$ worse than deviations below $\bar{p}$. (We also consider some "nicer" functions, like a Beta distributions, but we opted for a computationally simpler implementation.) Figure 3 shows the function f.

Figure 3: *MSGM* heuristic for population



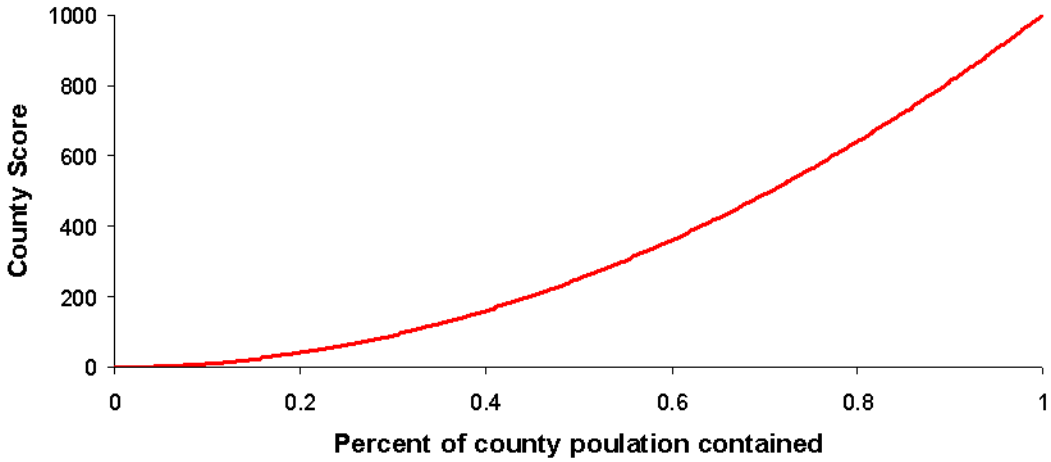
3.3 County preservation score

For a given district D_i, we measure its county preservation score in terms of the percent of counties that it completes on a population basis. To encourage growing districts to add remaining tracts in nearly complete counties the marginal value adding these should increase with the fraction of the population already contained in that district. To accomplish this we use the square of the proportion contained in a county. The county score, g, for a district D_i is:

$$g(D_i) = \sum_{C_j \in \Lambda} \left(\frac{\sum_{T_l \in D_i \cap C_j} p(T_l)}{p(C_j)} \right)^2 \tag{2}$$

For instance, if a district completely contains one county and contains 30% of each of two other counties' populations then its score would be $(1^2 + .3^2 + .3^2) = 1.18$. Figure 4 shows a plot of the county score a district receives based on what percent of a counties population said district contains.

Figure 4: *MSGM* heuristic for county completeness



4 The *Partition Optimization Model*

Now that we have constructed a crude, approximate solution to the districting problem by using *MSGM*, we refine the solution through a process of local search. We define our local search by our objective function, and our neighborhood function and search space.

4.1 The objective function

For our optimization function, the only characteristics of each district and each county we will use are the populations $p(P) = \{p_i\}_{1 \le i \le n}$, the compactness measures $c(P) = \{c_i\}_{1 \le i \le n}$, and the fractions $\rho(P) = \{\rho_{i,r} | 1 \le i \le n, 1 \le r \le c\}$ of the population of county r which is

contained in district i. Based on our analysis of desired properties of districts, we would like our score function $s(P) = s(p(P), c(P), \rho(P))$ to have the following properties:

1. the score function should be unimodal as a function of p_i, with mode at $p_i = \bar{p}$;
2. The score should increase more by adding tracts which lie in $\chi(D_i)$, so that we prefer having as few districts as possible in a given county.
3. The score should increase by more by adding tracts which increase the sum of all compactness measures by the greatest amount.

When considering these three constraints, they suggest that we should consider the three vectors $p(P), c(P), \rho(P)$ independently of each other in the score function, and then compare the scores of each when deciding on how to make tradeoffs between population equality, compactness, and county unity. In other words, we would like our score function to be a separable function of these three vectors, i.e. s has the form

$$s(P) = f(p(P)) + g(c(P)) + h(\rho(P))$$

where f, g, h are functions.

4.1.1 One (wo)man, one vote

Based upon the first criterion, we only require a globally concave down function whose maximum is attained at $p_i = \bar{p}$ for all p_i: $\frac{\partial s}{\partial p_i}|_{p_i=\bar{p}} = 0, \frac{\partial 2s}{\partial p_i 2} < 0$. The simplest functional form which satisfies this constraint is:

$$f(p(P)) = -\alpha_p \sum_{i=1}^{n} (p_i - \bar{p})^2$$

where α_p is some constant. That is, the score attributable to population differences is actually a constant multiple of the population variance across districts (once all tracts are assigned to a district).

The *MSGM* creates districts with approximate population equality by penalizing extreme variation away from $\bar{p}$ but equality is generally pretty weak. In one, more or less typical run of *MSGM* the districts created vary from 600,000 to 700,000, an unacceptable difference for a final districting plan.

By far, the most important constraint in determining district lines is that the populations within each district are very similar. Note that, this criterion is based on the *general population* within districts not the voting-age population or the population of likely voters.

Recall that our State has total population P and an average population of $\bar{p} = P/n$ per district. Letting p_i be the population in district i we consider three potential metrics for the population variance between districts.

1. Variance: $Var(p_1, p_2, \ldots, p_n)$
2. Maximum deviation: $\max\{|p_i - \bar{p}|\}$

3. Maximum difference: $\max\{p_i\} - \min\{p_i\}$

For all of these measures lower values are preferable and the minimum value is 0. We submit that choice number 1, variance, is the superior alternative. To see why, consider two possible population distributions between districts:

- *Situation A* - one district has a population of $1.05\bar{p}$, one is $.95\bar{p}$, and all of the others are $\bar{p}$
- *Situation B* - half of the districts have population $1.05\bar{p}$ and half have $.95\bar{p}$ (any left over odd district has $\bar{p}$)

In *Situation A* only two districts are different from the ideal population level, $\bar{p}$, but in *Situation B* very few districts have population $\bar{p}$ so a good metric should rank B worse than A. Clearly, the variance of populations is higher in B than in A, so variance passes this test. The maximum difference test gives $.05\bar{p}$ for both A and B and the maximum difference gives $.1\bar{p}$ for both.

We see that variance is the best measure of similarity since it factors in the pair wise difference in all district populations. We use variance as our measure of populational inequality between districts.

4.1.2 Compactness

To measure the compactness of a district we would ideally use our compactness measure:

$$c_i = \frac{Area(D_i)}{Perimeter(D_i)^2}$$

Such that:

$$g(c(P)) = \beta \sum_{i=1}^{n} c_i$$

where β is some constant.

Unfortunately, try as we might, we were unable to calculate the perimeter of tracts on the aggregate - the C++ library we used to interact with our census data shapefiles exhibited a variety of disturbing characteristics for different methods we used for calculating perimeters, including massive memory leaks for large-scale union operations, questionable accuracy for pairwise unions, and seemingly arbitrary calculations of intersection length.

Yet it is a poor craftsman that blames his tools and so undaunted, we adopted a different measure of compactness. Called the *clustering coefficient*, it provides a rough approximation for compactness. We define it as:

$$cc(D_i) = \frac{\sum_{T_l \in D_i} |\{T_k \in D_i | T_k \sim T_j\}|}{\binom{m_i}{2}}$$

such that:

$$g(c(P)) = \beta \sum_{i=1}^{n} cc(D_i)$$

where β is some constant. Our clustering coefficient thus provides a ratio of the total number of inter-district boundaries to the maximum possible number of inter-district boundaries. Note that if all tracts were uniformly shaped, this measure would prize square- and circle-shaped districts, while winding, single tract-width districts would be penalized. However, given the asymmetry of tract shapes, this measure does little to reflect negatively upon district shapes such as the dumbbell, two circular clusters of tracts connected by a narrow band of tracts. In general however, the clustering coefficient will value adding to districts tracts that are "close" and removing from districts those tracts that are auxiliary.

4.1.3 County preservation

We adopt the same county preservation measure used in the *MSGM*, defined in equation 2 with the option of adding a scaling factor to the entire function to refine empirical performance.

4.2 Search method and neighborhood function

In order to refine our solution from *MSGM*, we must move tracts between districts. Yet the space of all possible contiguous moves is too large to run effectively. We solve this problem considering a range of possible moves with respect to only one district, its boundary and frontier, and performing the best move on this dramatically reduced state space.

By selecting our target district at random at each iteration, our strategy is best described as *stochastic domain hill climbing*. It is a method that combines the best aspects of both random and deterministic local search methods - we perform optimal moves while avoiding getting stuck trying to only increase the score of a single district. After determining that simple first-order moves on the district level, that is, adding or removing individual tracts, were incapable of reducing our variance metric to the extremely low standard that was our charge, we expanded our search to include second-order moves, that is, "swaps", a combined move that includes both an add and remove within a single operation.

If we assume that the maximum connectedness of any tract on the graph is k, checking for all adds and removes separately for district D_i involves considering $O(k|\partial D_i| + |F_i|) = O(km_i)$ possible moves, while looking at all swaps involves considering $O(k|\partial D_i||F_i|) = O(km_i^2)$ possible moves. One might contend, then, that the operation of checking *every* district for first-order moves might be a better algorithm, as it would take $O(\sum_{i=1}^{n} km_i) = O(nkm_i)$ heuristic evaluations. One could even supplement such an algorithm with a degree of randomness, to avoid being caught in a possible loop of futility, by employing simulated annealing, stochastic hill climbing, or tabu search on the resulting list of possible future states. In practice, however, we found that checking for second-order moves provided far better empirical results with acceptable time performance, while an algorithm enumerating all the possible second-order states, requiring $O(\sum_{i=1}^{n} km_i^2) = O(nkm_i^2)$ heuristic evaluations, was too slow to be effective.

The true heart of *POM* is the following algorithm. For simplicity and readability, we let $M_{add}(D_i)$ be the set of all moves in which we add a frontier tract to D_i, and $M_{remove}(D_i)$ to be the set of all moves in which we remove a border tract from D_i, and M^{-1} the move that is the inverse of M, such that applying both M and M^{-1} in turn has no effect. Recall also that our heuristic scores partition P as $s(P)$.

Input: Iteration count *iter*, initial partition P.
Output: Final partition P.
$count \leftarrow 0$
while $count < iter$ **do**
  $curscore \leftarrow s(P)$
  $D \leftarrow randomDistrict()$
  $bestscore \leftarrow curscore$
  foreach $M_a \in \{\emptyset \cup M_{add}(D)\}$ **do**
    foreach $M_r \in \{\emptyset \cup M_{remove}(D)\}$ **do**
      $performMove(M_a)$
      $performMove(M_r)$
      if $isContiguous(P)$ **then**
        $tmpscore \leftarrow s(P)$
        if $tmpscore > bestscore$ **then**
          $bestscore \leftarrow tmpscore$
          $bestadd \leftarrow M_a$
          $bestremove \leftarrow M_r$
        end
      end
      $performMove(M_a^{-1})$
      $performMove(M_r^{-1})$
    end
  end
  if $bestscore > curscore$ **then**
    $performMove(bestadd)$
    $performMove(bestremove)$
  end
  $count \leftarrow count + 1$
end
return P

Algorithm 1 - Stochastic domain hill-climbing algorithm for districting

Note that we guarantee that our solution will be contiguous by not even considering moves that would break contiguity, and that we only perform a move if it increases the score of our current state.

4.3 Achieving absolute equality

US law mandates that the populations of each district be equal within a range of error of one person according to the census data (Karcher v. Daggett, 1983). Our problem dealt only with census tracts, and so exact equality of populations to the nearest integer was not possible. This last step of the algorithm must be implemented by splitting tracts between two districts.

To the knowledge of the authors, this problem beyond population unit level (no smaller than block groups) has not been addressed in the literature. Clearly, the simplest way to

do this is to split one of the border tracts. While we do not implement this part of the algorithm in the computer simulation, we describe the methodology for doing this.

Let G denote the graph whose vertices are given by the districts and whose edges are the pairs of bordering districts. The intuition for the algorithm is that if we can find a pair of districts such that splitting a border tract between them gives both districts a population of one within the mean population, then we would optimally do so and ignore those two districts for the remainder of the algorithm. However, to guarantee that the algorithm finishes, we require that the graph G remain connected (otherwise, G may divide into two or more connected components, such that the constituent districts cannot attain populations equal to the overall mean). Taking out two districts at a time by splitting only a single tract leaves the fewest possible tracts split, which we consider optimal, for the same reasons that number of counties split between districts is optimal.

Our algorithm works as follows. We search for an edge of G such that removal of its two vertices and all edges emanating from them leaves a new graph $G_1 \subset G$ that is connected. We call the deletion of a single vertex from a graph that leaves the graph connected a *paring*. If these two vertices have some special properties, we perform the double paring and then perform the algorithm on G_1, and continue until all districts have equal population. If no such pair of districts exists, we then perform a single paring and ensure that the removed district has population $\bar{p}$ before removing it. Define *tract splitting* to be the process of splitting up a border tract into two disjoint areas and two disjoint populations allocated between two bordering districts.

There always exists an edge on a connected graph G that permits a double paring of G, except for a very specialized set of connected graphs. However, all connected graphs permit a paring, as the next theorem shows.

Theorem 4.1 *All connected graphs permit a paring.*

A proof of this theorem is given in the Appendix B.

We recursively update the districts to get population equality. We iteratively pare the graph G of districts such that each time we pare a district or pair of districts, those districts have populations which equal the population mean. By Theorem 4.1, this process always ends with all districts having equal population. Our algorithm works as follows:

1. If the graph G contains only one district, its population must equal $\bar{p}$. Stop the algorithm here. If not, search across all border tracts of the partition for a tract such that splitting it between two districts makes the population of the two border tracts within 1 of the average $\bar{p}$. If some pair of districts exists which is a double paring of G, then perform this double paring of G. For these two districts, take the tract on their border which, upon being split between the two districts, makes their populations within 1 of the population mean. Split this tract to equalize their populations. If no such pair exists, go to Step 2.

2. Search G for all possible double parings such that the two districts in the double paring have populations which sum to twice the average population. Perform the double paring of G among these double parings which has the property that the two removed districts can have equal populations with the minimal number of tract moves

and one tract splitting between the two. If such a pair exists, perform the double paring and go to Step 1. If no such pair exists, go to Step 3.

3. Search all vertices of G for a paring of G such that a single tract splitting along the border of the district gives the district a population of *barp*, and perform this paring of G. If such a border tract and paring exist, perform the paring and the tract splitting, and go to Step 1. If no such tract splitting and paring exist, go to Step 4.

4. Search all vertices of G for a paring of G such that the removed district D_i borders a district which requires the minimum number of moves and one tract splitting to make the population of D_i equal to $\bar{p}$. Perform these moves, this tract splitting, and this paring, and return to Step 1.

This entire algorithm removes at least one vertex from G at each steps, and the whole algorithm can therefore be performed with at most $n-1$ tract splittings, where n is the number of districts. The actual number of tract splittings equals $n-d-1$, where d is the number of double parings performed.

5 Case Study: New York congressional districts

5.1 The data

We began our inquiry by acquiring data from the 2000 census from the New York State Data Center. The downloaded data contained 4907 tracts, but a number of these were tracts have no population. These tracts represented water, inland lakes, or parks. We considered all of these tracts to be the equivalent of water, with the exception of only one of these tracts on Long Island which completely enclosed a populated "island" and was thus considered to be a tract of land with no population. These empty districts are the cause of the "holes" on our maps, particularly around the NYC Metro area.

Trimming these parts from our map left us with 4827 tracts to examine. It is worth noting that the possible number of partitions of these tracts is prohibitively high. Ignoring concerns such as contiguity, nonempty districts, or population equality, the number of allocations of 4827 tracts to 29 districts is approximately

$$\frac{1}{29!}29^{4827} \approx 1.1 \times 10^{7028}$$

The data were delivered in ESRI shapefile format, which listed tract areas, populations, and unique county identifiers.

5.2 Results

Running the *MSGM* on our initial allocation left us with 29 haggard districts spanning the map from which to refine a solution.

Using this solution as a starting point, we optimized our result using swap moves in particular. Though our algorithmic process of refinement is stochastic, generally more than 90% of the moves in any run involved swaps. This was particularly the case for moves

Table 2: Values after the *MSGM*

Variable	Value
Heuristic Variance Score	-3,147
Largest District	969,511
Smallest District	280,945
County Score	37.48
Compactness Score	2,869

at the very end of a run, where population differences between districts were minute. As a result, swapping provided a way to adjust population smoothly. In addition swap operations, particularly of side-by-side tracts exchanged between districts, provided an effective to "clean up" tattered fringes of districts, increasing their compactness even with vigorous population changes.

Table 3: Values after refinement

Variable	Value
Heuristic Variance Score	-.0277
Largest District	655,760
Smallest District	652,561
County Score	47.44
Compactness Score	2,906

The most difficult part of both steps was defining the optimal values for the scaling factors. It is important to note that it is not the magnitude of the scaling factors that is most crucial, but rather their *relative marginal* magnitudes. Since our algorithm operates on the changes that result from making a single first- or second-order move, selecting positions with the highest score, it is important that the changes in each of the heuristic variables are significant. In particular, a large or small multiple on some factor does not indicate that we wished to treat that variable severely or lightly, but rather that the marginal changes in that variable were relatively small or large.

The Appendix contains several informative tables and maps summarizing our results. Images are produced using the amazing TatukGIS Viewer software.

6 Extension: The 4th Dimension

It is entirely possible that a state's congressional districts could become populationally imbalanced between redistrictings, which usually occur every 10 years. Though current practice is to devise a districting with equal populations per district we suggest that this is suboptimal. One could imagine an initial population allocation that maximizes district population equality not just in the first years but over the course of all 10 years between redistrictings.

For instance, if one district's population is growing 2% a year and another's is shrinking 1% a year then after ten years the two populations will differ by over 33%. With congressional elections occurring every two years it seems arbitary to privilege the population at the year 2000 rather than at the years 2002, 2004, etc. . To improve this disparty we propose starting the growing district with a slightly lower population than that of the shrinking district.

6.1 A stitch in time

For each tract, we can observe certain demographic characteristics, such as race. Based upon population growth estimates from the Census Bureau we can find optimal weighting of populations such that citizens do not have an "equal vote" today, but citizens have the most equal vote over the entire period between each redistrictings.

Let T denote the time between redistrictings; in our case $T = 10$ because the census is taken decenially in the United States. In this section we explore the effect of differential population growth rates by districts on optimal population weights for the districts.

Modern utility theory suggests that individuals favor present utility greater than future utility, and most often, for analytical convenience, according to a constant time discount factor. Let us suppose that the time discount factor for utility of individuals in the United States is given by δ.

We assume that societal utility is maximized by giving citizens an equal voting share in each period. (If this does not actually maximize utility then one could still argue that *ideal* politicians would prefer a scheme that promotes voting share equal.) As we discussed in Section 4.1.1, variance is the best measure for population inequality between districts.

Utility today is weighted greater than utility t units in the future by a factor of $e^{\delta t}$. If we have a partition $\Omega = \{D_1, ..., D_n\}$, with populations $p_1, ..., p_n$, then the population penalty we found for such a partition is a constant multiple of $Var(p_i)$. Let $p_{i,t}$ denote the population of district i at time t. Then the discounted utility of the state at time t is $e^{-\delta t}Var(p_i)$. Suppose that we have forecast data on the population growth rates of different counties during the T-year period. Let the log-growth rate at time t for district i be given by $\eta_{i,t}$. Then the population of district i at time t is given by:

$$p_{i,t} = \exp\left(\int_0^t \eta_{i,s} ds\right) p_{i,0}$$

and total utility of the initial allocation Ω with district populations $\mathbf{p} = (p_1, p_2, ..., p_n)'$ is

$$U_{[0,T]}(\Omega) = -\int_0^T e^{-\delta T} \mathrm{Var}(p_{i,t}) dt$$

Expressing the variance in terms of the populations $p_{i,t}$, we get

$$\mathrm{Var}(p_{i,t}) = \frac{1}{n}\sum_{i=1}^n p_{i,t}^2 - \frac{1}{n^2}\left(\sum_{i=1}^n p_{i,t}\right)^2 = \frac{n-1}{n^2}\sum_{i=1}^n p_{i,t}^2 - \frac{1}{n^2}\sum_{i\neq j} p_{i,t}p_{j,t}$$

Dividing out by a constant factor, this gives the total utility as

$$\begin{aligned} U_{[0,T]}(\Omega) &= -(n-1)\sum_{i=1}^{n} p_i^2 \int_0^T \exp\left(2\int_0^t \eta_{i,s}ds - 2\delta t\right)dt \\ &\quad + \sum_{i\neq j} p_i p_j \int_0^T \exp\left(\int_0^t (\eta_{i,s}+\eta_{j,s})ds - 2\delta t\right)dt \end{aligned}$$

This functional form is convenient if we choose to give a specific stochastic process which the logarithmic growth rate may follow. Since our time period is relatively short, we will assume that population growth is simply exponential and thus log-growth rates are constant within our time window, 10 years. So, we set $\eta_{i,s} = \eta_i$ and also define the time-discounted population growth as $\nu_i \equiv \eta_i - \delta$. As long as $\nu_i + \nu_j \neq 0$ then utility simplifies to:

$$U_{[0,T]}(\Omega) = -(n-1)\sum_{i=1}^{n}\left(p_i^2\int_0^T e^{2(\eta_i-\delta)}dt\right) + \sum_{i\neq j}\left(p_i p_j \int_0^T e^{[(\eta_i-\delta)+(\eta_j-\delta)]t}dt\right)$$

$$= -(n-1)\sum_{i=1}^{n}\left(\frac{e^{2\nu_i T}-1}{2\nu_i}p_i^2\right) + \sum_{i\neq j}\left(\frac{e^{(\nu_i+\nu_j)T}-1}{\nu_i+\nu_j}p_i p_j\right)$$

.

We define the optimal vector of target populations as $\mathbf{p}^* = (p_1^*, ..., p_n^*)^T$ where p_i^* is the optimal population for district i. Under the constraint $\sum_i p_i = P$ (the population of the whole State) we use Lagrange Multipliers to obtain:

$$\lambda = \frac{\partial U_{[0,T]}(\Omega)}{\partial p_i} = -(n-1)\frac{e^{2\nu_i T}-1}{\nu_i}p_i + \sum_{j\neq i} 2\frac{e^{(\nu_i+\nu_j)T}-1}{\nu_i+\nu_j}p_j, 1 \leq i \leq n$$

.

It follows that the vector $\mathbf{p}^*$ satisfies

$$\mathbf{H}\mathbf{p}^* = \lambda\iota$$

,

where $\mathbf{H}$ is the matrix of coefficients

$$\mathbf{H} = \begin{pmatrix} -(n-1)\frac{e^{2\nu_1 T}-1}{\nu_1} & 2\frac{e^{(\nu_1+\nu_2)T}-1}{\nu_1+\nu_2} & \dots & 2\frac{e^{(\nu_1+\nu_n)T}-1}{\nu_1+\nu_n} \\ 2\frac{e^{(\nu_2+\nu_1)T}-1}{\nu_2+\nu_1} & -(n-1)\frac{e^{2\nu_2 T}-1}{\nu_2} & \dots & 2\frac{e^{(\nu_2+\nu_n)T}-1}{\nu_2+\nu_n} \\ \vdots & \vdots & \ddots & \vdots \\ 2\frac{e^{(\nu_n+\nu_1)T}-1}{\nu_1+\nu_n} & 2\frac{e^{(\nu_n+\nu_2)T}-1}{\nu_2+\nu_n} & \dots & -(n-1)\frac{e^{2\nu_n T}-1}{\nu_n} \end{pmatrix}$$

where $\iota = (1, 1, ..., 1)'$ is an $n \times 1$ vector of ones and λ is the Lagrange multiplier.

The expression for $\mathbf{H}$ is analytically convenient as $\mathbf{H}$ is symmetric, and by the Spectral Theorem is orthogonally diagonalizable, enabling a computationally feasible inversion of $\mathbf{H}$ to solve for the optimal populations $\mathbf{p}^*$:

$$\mathbf{p}^* = \lambda\mathbf{H}^{-1}\iota$$

This uniquely determines λ, as the sum of the components of $\mathbf{p}^*$ must be P. We get

$$\lambda = \frac{P}{\iota' \mathbf{H^{-1}} \iota}$$

and this yields the final formula

$$\mathbf{p}^* = \frac{P}{\iota' \mathbf{H^{-1}} \iota} \mathbf{H^{-1}} \iota \tag{3}$$

In the actual implementation, the growth rate η_i is such that if the annual growth rate is g_i, then we have $1 + g_i = e^{\eta_i}$, or

$$g_i = e^{\eta_i} - 1$$

While the estimation of δ is not purely objective, it is reasonable to set the discount rate equal to the discount rate of consumption. In utility-theory analysis, the best measure of the discount rate of consumption is the risk-free interest rate, which is currently best approximated by the overnight lending rate set by the United States Federal Reserve Bank, which is at an annualized $r = 5.25\%$. This implies that if the discount rate is δ, then δ is given by $e^\delta = 1 + r$, or

$$\delta = \log(1 + r) \approx 5.1168\%$$

We use this rough approximation in the following section.

6.2 Implementation of the extension

We are using data from the 2000 census, so to estimate the population growth rates in the 2000-2010 redistricting period, we use realized *county* population growth rates during the 2000-2003 period.

The output of our model gives allocations based on equal population and we estimate the population growth rates of the *districts* by assuming uniform population growth rates within each county. It is easy to calculate how much each district is made up of various counties and we use these proportions as weights to approximate the *district population growth rate* as a weighted average of *county population growth rates*.[1]

Based on the optimal population vector $\mathbf{p}^*$ found via Equation (3) we can rerun *POM* with the populations goal of $\mathbf{p}^*$. This procedure can be iterated as: run the *POM*, find the growth rates of each district produced, calculate the optimizations of initial populations based on the above theory, and feed the results back into *POM*. We settle on a final districting plan when the solution converges within some reasonable bound.

Figure 4 shows one iteration of this process. The initial result from *POM* is p_i and district growth rates are found using our Census data about county growth rates. The final column shows the optimal initial population that from Equation (3) that will maximize societal voting equality over the entire period between redistrictings. One can easily see

[1]We are assuming that district growth rates remains constant over time which is inconsistent with our previous assumption that the county growth rates are constant. This is a small, simplifying assumption and the interested reader may make these assumptions consistent by explicitly calculating district growth rates over time in terms of the county growth rates and initial population distribution of counties in districts. The theory above, using stochastic logarithmic growth rates, is designed to accomodate such generalizations.

that districts with higher projected growth rates (η_i) are assigned lower optimal starting populations.

The results make intuitive sense: faster growing districts are initially under-allocated and slower growing districts are over-allocated in terms of starting population. There is a significant effect of taking into account population changes over time. The difference between the smallest and largest optimal district populations is 69,133, which is 10.6% of the total average district population. This implies that, with a reasonable level of certainty about future population growth rates, it may be beneficial for legislators to take future population growth into account when redistricting.

Table 4: District Population Growth Rates

p_i	Est. 2003 population	η_i	Optimal initial pop.
655,067	681,997	2.01%	613,786
654,373	678,814	1.83%	618,869
655,760	678,245	1.69%	622,818
654,715	673,058	1.38%	631,544
654,449	668,395	1.05%	640,802
654,140	667,976	1.05%	640,802
655,184	668,555	1.01%	641,922
653,486	666,411	0.98%	642,761
653,636	665,372	0.89%	645,278
654,702	665,452	0.81%	647,513
654,164	664,307	0.77%	648,630
653,902	663,066	0.70%	650,582
653,884	662,672	0.67%	651,418
652,561	659,850	0.56%	654,482
655,040	660,798	0.44%	657,818
654,383	659,926	0.42%	658,374
653,655	656,265	0.20%	664,474
655,311	657,351	0.16%	665,581
653,676	655,585	0.15%	665,857
653,792	655,701	0.15%	665,857
654,568	656,471	0.15%	665,857
654,739	656,443	0.13%	666,411
654,745	655,765	0.08%	667,793
654,041	654,476	0.03%	669,173
654,834	654,665	-0.01%	670,277
654,381	654,019	- 0.03%	670,829
654,242	653,298	-0.07%	671,932
654,395	648,670	-0.44%	682,097
654,632	648,514	-0.47%	682,919

In the above, p_i is the value that our model returns for the population of the 29 districts. The estimated 2003 populations are calculated for each district based on county growth rates. One can easily see that districts with higher projected growth rates (η_i) are assigned lower optimal starting populations.

7 Analysis of the Models

7.1 Solving the Problem

By combining the *Multi-seeded Growth Model* with the *Partition Optimization Model* we effectively devised a strategy for creating fair and geometrically compact congressional districts. The districts conform to several well accepted measures of district goodness: population equality, contiguity, preservation of county boundaries, and compactness of shape.

The districts produced by our models are both simple and fair. Geometric *simplicity* is measured by compactness, as determined by how close the members of a districts live realtive to each other. Additionally, our method penalizes splitting counties between several districts so that nearby citizens, who have simliar concerns, will be represented by the same congressperson. The *fairness* of our methodology is evident in its perfect indifference to partisan politics, incumbent protection, and race/ethnicity.

We apply our models to create a congressional district partition of New York State based on 2000 US Census Bureau data. The results in Figures 6, 8, and 10 clearly demonstrate a partitioning into contiguous, compact, and reasonable districts. Furthermore, the simulations that produced these visually pleasing results also achieved extremely high degrees of population equality and county preservation.

7.2 Strengths of Model

The model successfully generates district partitions that simultaneously excel against the standard metrics of county integrity, compactness, and population equality. Unlike other models in the literature, we provide an algorithm for reducing population differences to at most 1 by breaking up a minimal number of tracts.

We also find that in order to equalize population of the districts as much as possible, any knowledge about future district growth rates yields highly unequal initial district populations, contrary to one of the fundamental assumptions of all existing algorithms in the literature.

The model runs independently of the distribution of population, and works well both in low- and high- density locales, and with regular and oddly shaped census tracts. This is evidenced by the successful districtings that our model produces in rural, small city, and large metropolitan areas. (See the Figures 5 through 10.)

The algorithm runs efficiently enough that it can generate districts for large States, such as New York (population: 18,976,457), in a run time of less than an hour.

7.3 Weaknesses of Model

The model assumes contiguity of the entire State so in cases where contiguity cannot be forced, such as Hawaii or Michigan, we must change the algorithm slightly. One solution could be to divide the State into several regions and run our model separately on each region, allocating the porportionally correct number of representatives to each region based on population.

A second limitation is that the model appears to tend toward creating districts that are either very low- or high-density, instead of splitting smaller population centers into a number

of districts. As political affiliation and race are likely correlated with population density, the algorithm may inadvertently generate districts which separate various demographic groups into separate districts, which could be viewed as gerrymandering. Yet, another camp would argue that it is appropriate to divide urban, suburban, and rural areas into separate districts since their residents have different concerns.

7.4 Future Investigations

A problem with any computer-based solution to the redistricting problem is that the methodology used in the redistricting algorithm may indirectly lead to some form of gerrymandering. Because the program is not deterministic and can be evaulated many times, the entity running the program should not be able to arbitrarily choose a result as this could be characterized as gerrymandering. (We tie our hands by choosing the highest scoring result based on our goodness metric but a future modeller with an ulterior motive could be less objective.)

To solve this we should test our simulations and throw out any results that, by random chance, display the qualities of partisan or racial/ethnic gerrymandering. This could be done relatively easily by merging tract level data with data political and racial characteristics.

This model sought to create a baseline alternative to the political misuse of congressional districting, but it could be expanded to a loftier goal. For instance, we assume that race/ethnicity should play no role in creating districts but it is conceivable that citizens are better off when minority groups control a few districts so that these groups are guaranteed at least a few representatives. If every district is a perfect cross-section of the State's demographics then minority groups will have *ex ante* equal political power but not *ex post*. More work needs to be done to understand the legal, philosophical, and mathematical underpinnings of districting in a representative democracy.

An open letter concerning congressional districting

TO: Sheldon Silver, Assembly Speaker, New York State Assembly
CC: Robert D. Lenhard, Chairman, Federal Election Comission
CC: Rex Smith, Editor, Albany Time Union
FROM: MCM Team # 1421
DATE: February 12, 2007

The negative consequences of Gerrymandering are well accepted: voters become apathetic, minority groups are sequestered to a few districts, and the political process moves farther and farther from the electorate's best interests. We present to the you, the Assemblymen and Assemblywomen of New York, a new method to create fair districts with simple shapes that citizens will appreciate and embrace.

We have devised a set of rules that a computer can implement to create districts that are:

1. Contiguous - there are no breaks in the district lines
2. Equally sized in population
3. Conscious of county boundaries - especially in upstate New York congressional districts will avoid splitting county lines
4. Compact - districts are not winding, long and skinny, or oddly shaped

Our scheme produces fair districts in that choices are made without prejudice or favor to residents of particular racial, ethnic, or socioeconomic groups. At the same time, by producing districts that break up the fewest possible tracts, we ensure that voters with roughly similar characteristics and geographical location will be represented by the same congressperson. This has the effect of encouraging civic involvement by residents, aligning representatives' interests with those of their consituents, and fostering a healthier democracy.

By implementing our redistricting method, the Empire State can be a pioneer in guaranteeing the rights of its citizens. Since the 19th Century, Elbridge Gerry's lizard has grown into a terrible, twisting serpent, eating away at our Democracy.

It is time to put Gerrymanders on a healthier diet.

References

[1] Barkan, J. D., P. J. Densham, and G. Rushton (2006). Space Matters: Designing Better Electoral Systems for Emerging Democracies. *American Journal of Political Science, 50* (4), 926-939.

[2] Bong, C. and Y. Wang (2006). A multi-objective hybrid metaheuristic for zone definition procedure. *Int. J. Services Operations and Informatics (1)* (1/2), 146-164.

[3] Caliper Corporation. "About Census Summary Levels." Available at http://www.caliper.com/Maptitude/Census2000Data/SummaryLevels.htm.

[4] Cirincione, C., T. A. Darling, and T. G. O'Rourke (2000). Assessing South Carolina's 1990s Congressional Redistricting. *Political Geography, 19*, 189-211.

[5] Garfinkel, R. S. and G. L. Nemhauser (1970). Optimal political districting by implicit enumeration techniques. *Management Science, 16* (4), B495-B508.

[6] Hunt v. Cromartie, 526 U. S. 541 (1999).

[7] Karcher v. Daggett, 462 U.S. 725 (1983).

[8] Kaiser, H. (1966). An objective method for establishing legislative districts. *Midwest Journal of Political Science, 10*.

[9] League of United Latin American Citizens v. Perry, 548 U.S. _____ (2006).

[10] Luttinger, J. M. (1973). Generalized Isoperimetric Inequalities. *Proceedings of the National Academy of Sciences of the United States of America, 70*, 1005-1006.

[11] Macmillan, W. (2001). Redistricting in a GIS environment: An optimisation algorithm using switching-points.

[12] Macmillan, W. and T. Pierce (1994). Optimization modeling in a GIS framework: the problem of political districting. In S. Fotheringham and P. Rogerson, *Spatial Analysis and GIS*. Bristol: Taylor and Francis. *Journal of Geographical Systems, 3*, 167-180.

[13] Mehrotra, A., E. L. Johnson and G. L. Nemhauser (1998). An Optimization Based Heuristic for Political Districting. *Management Science, 44* (8), 1100-1114.

[14] Miller v. Johnson, 515 U. S. 900 (1995).

[15] NationalAtlas.gov Maps of US Congressional Districts.

[16] Nagel, S. (1965). Simplified bipartisan computer redistricting. *Stanford Law Review, 17*, 863-899.

[17] ew York State Data Center. "2000 Census Data." Available at http://www.empire.state.ny.us/nysdc/.

[18] Shaw v. Reno , 509 U. S. 630 (1993).

[19] SC State Conference of Branches, Etc. v. Riley (1982). 533 F. Supp. 1178 (DSC). Affirmed 459 US 1025.

[20] S Census Bureau. "New York county data 2000-2003." Available at http://www.epodunk.com/top10/countyPop/coPop33.html.

[21] Weaver, J. B. and S. W. Hess (1963). A procedure for nonpartisan districting: development of computer techniques. *The Yale Law Journal, 73* (1), 287-308.

[22] ahoo Finance. Market data on US Treasury bond rates. Available at http://finance.yahoo.com/bonds/composite_ bond_ rates .

[23] Young, H. P. (1988). Measuring the Compactness of Legislative Districts. *Legislative Studies Quarterly.* **XIII** 105-115.

A Tables and Maps

Table 5: Final partition of counties after the *POM*. f is the fraction of the county allocated to the largest district in that county, while d represents the number of the districts with tracts in that county.

County Name	f	d	County Name	f	d
Albany	0.84	2	Niagara	1	1
Allegeny	1	1	Oneida	1	1
Bronx	0.74	4	Onondaga	0.94	2
Broome	0.71	2	Ontario	1	1
Cattaraugus	0.53	3	Orange	0.85	2
Cayuga	0.94	2	Orleans	1	1
Chautauqua	1	1	Oswego	0.92	2
Chemung	0.52	3	Otsego	0.59	2
Chenango	0.83	3	Putnam	1	1
Clinton	1	1	Queens	1	1
Columbia	0.9	2	Rensselaer	0.87	3
Cortland	1	1	Richmond	1	2
Delaware	0.56	2	Rockland	1	1
Dutchess	1	1	Saratoga	1	1
Erie	1	1	Schenectady	0.83	2
Essex	1	1	Schoharie	0.93	2
Franklin	1	1	Schuyler	0.64	6
Fulton	0.55	5	Seneca	1	1
Genessee	0.43	9	St. Lawrence	1	1
Greene	0.33	13	Steuben	1	1
Hamilton	0.53	7	Suffolk	0.81	2
Herkimer	1	1	Sullivan	1	1
Jefferson	0.42	9	Tioga	0.86	2
Kings	0.27	13	Tompkins	1	1
Lewis	0.88	3	Ulster	0.92	2
Livingston	1	1	Warren	0.71	2
Madison	0.75	2	Washington	0.6	4
Monroe	1	1	Wayne	0.51	4
Montgomery	1	1	Westchester	0.73	4
Nassau	1	2	Wyoming	1	1
New York	0.97	2	Yates	1	1

Averages: $f = .85$, $d = 2.55$

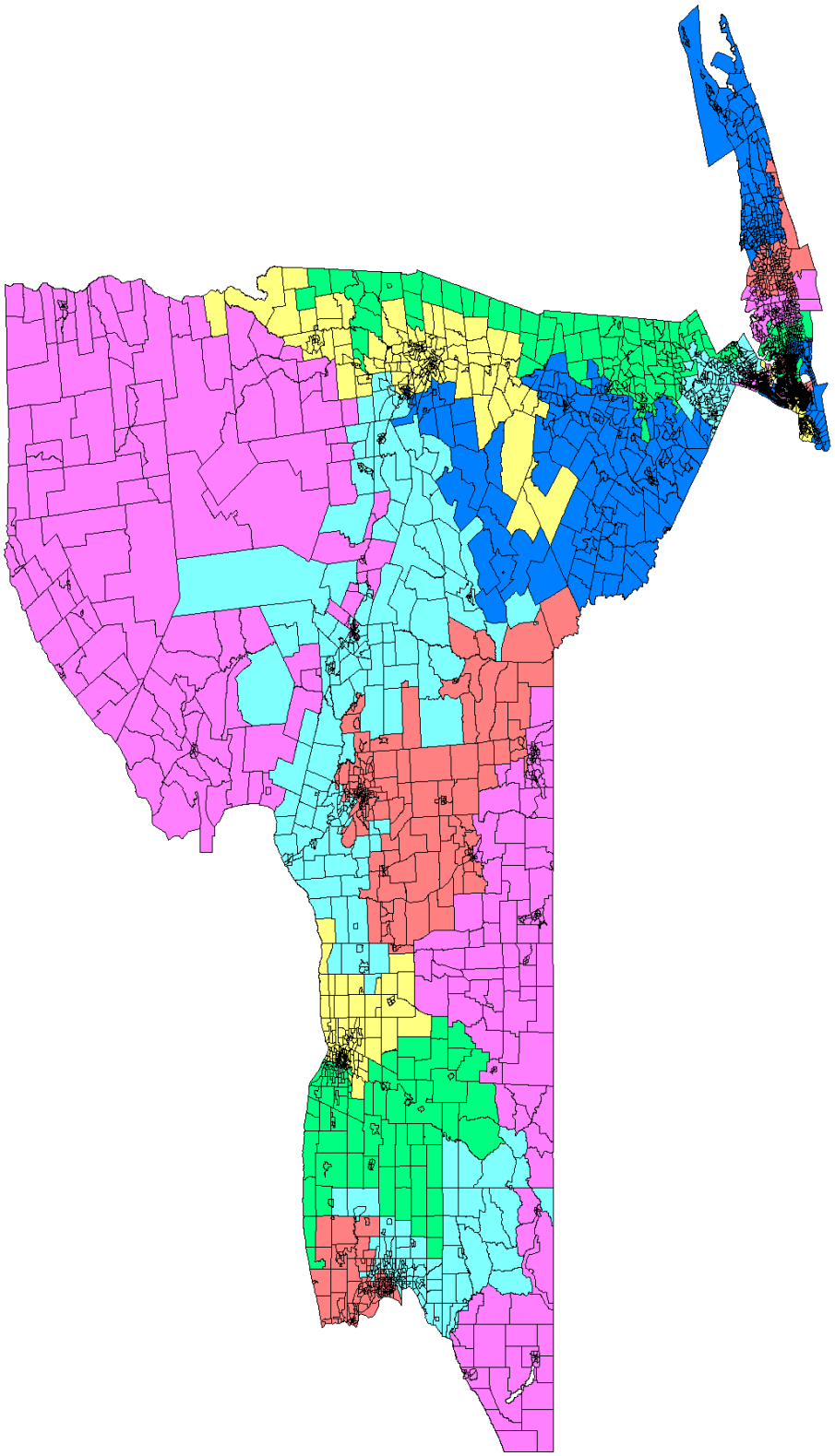


Figure 5: New York congressional districts from the *MSGM* (initialized districts)

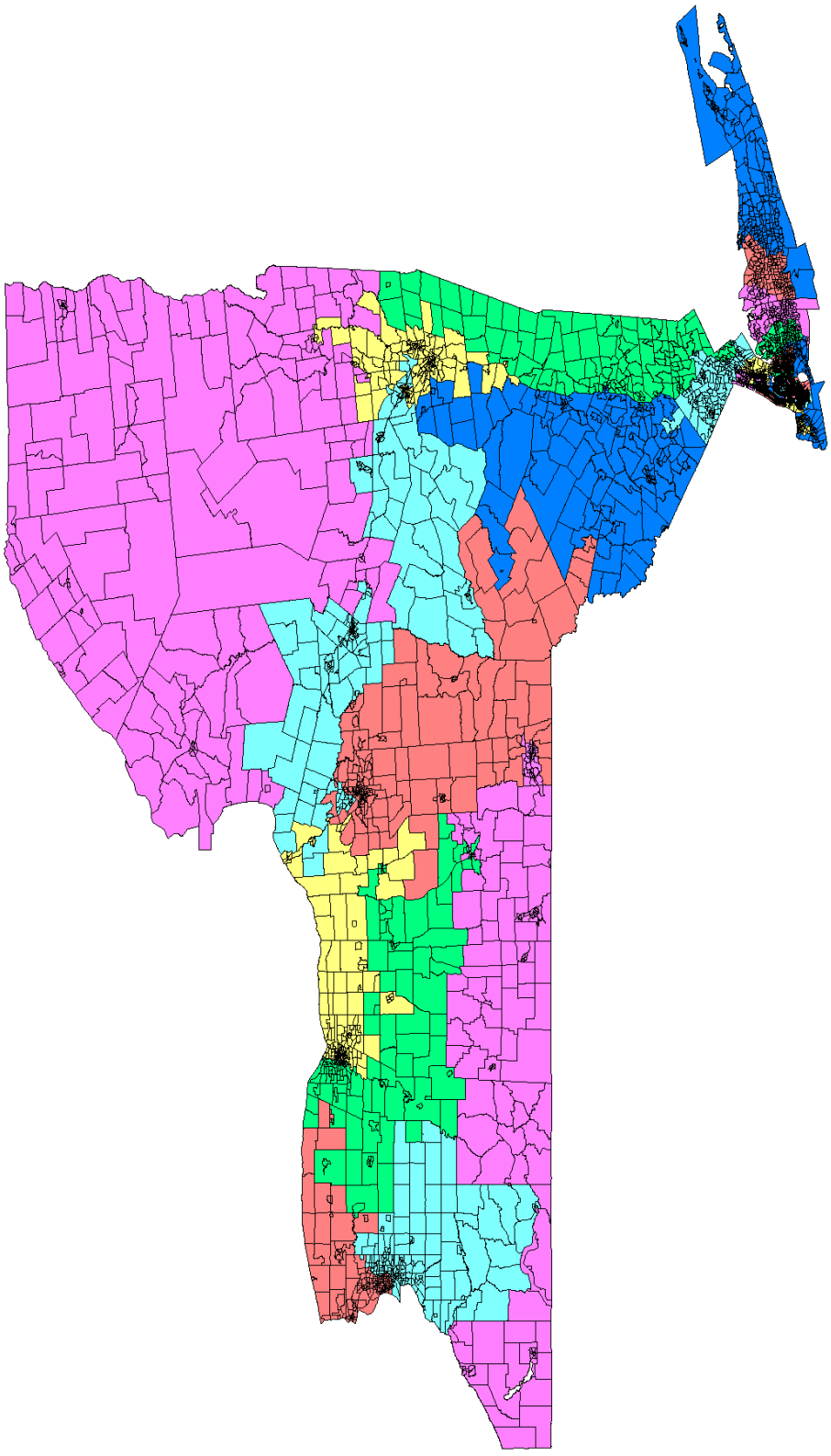


Figure 6: New York congressional districts from the *POM* (final optimization)

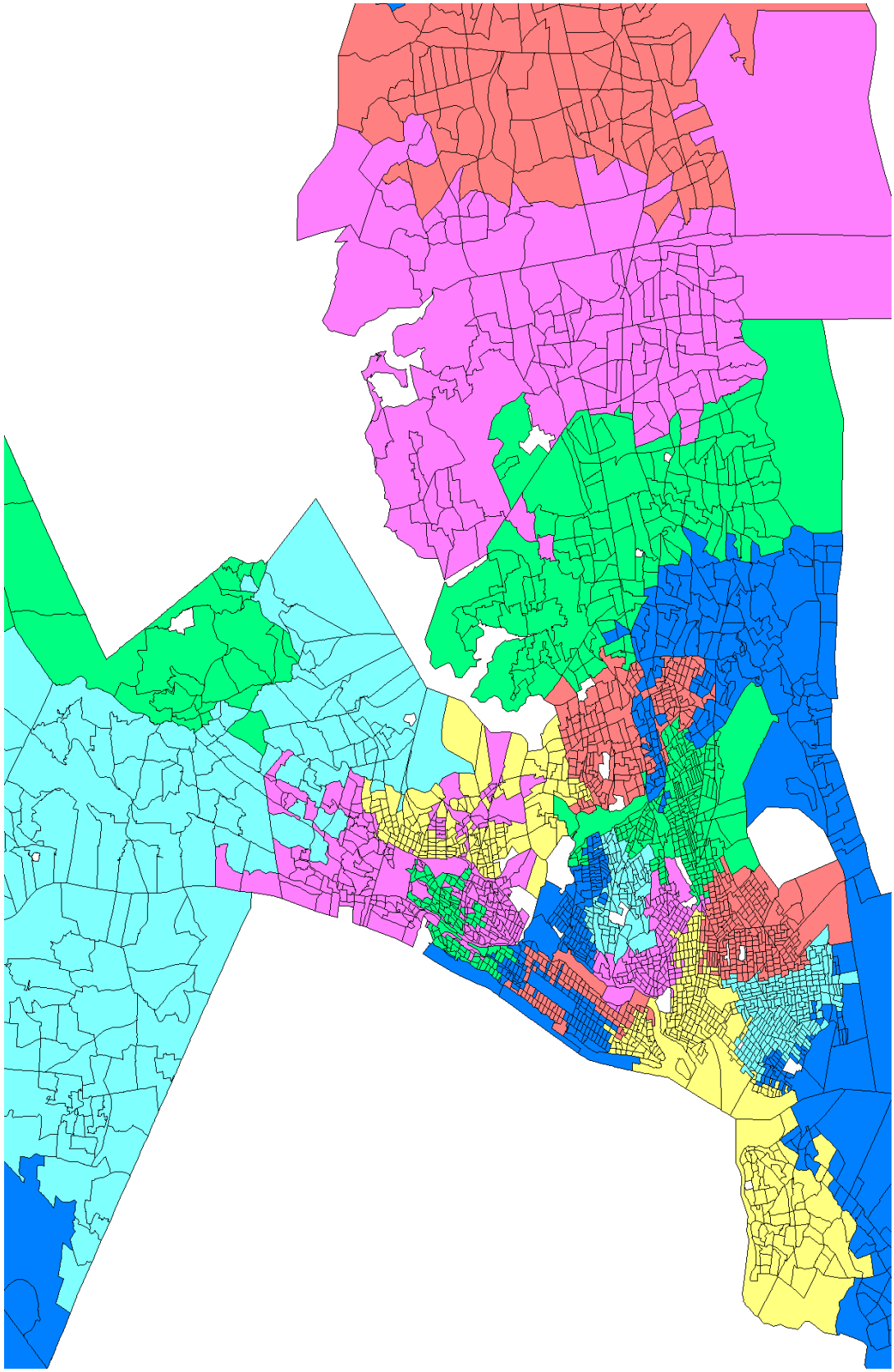


Figure 7: NYC metro-area $MSGM$ (initialized districts)

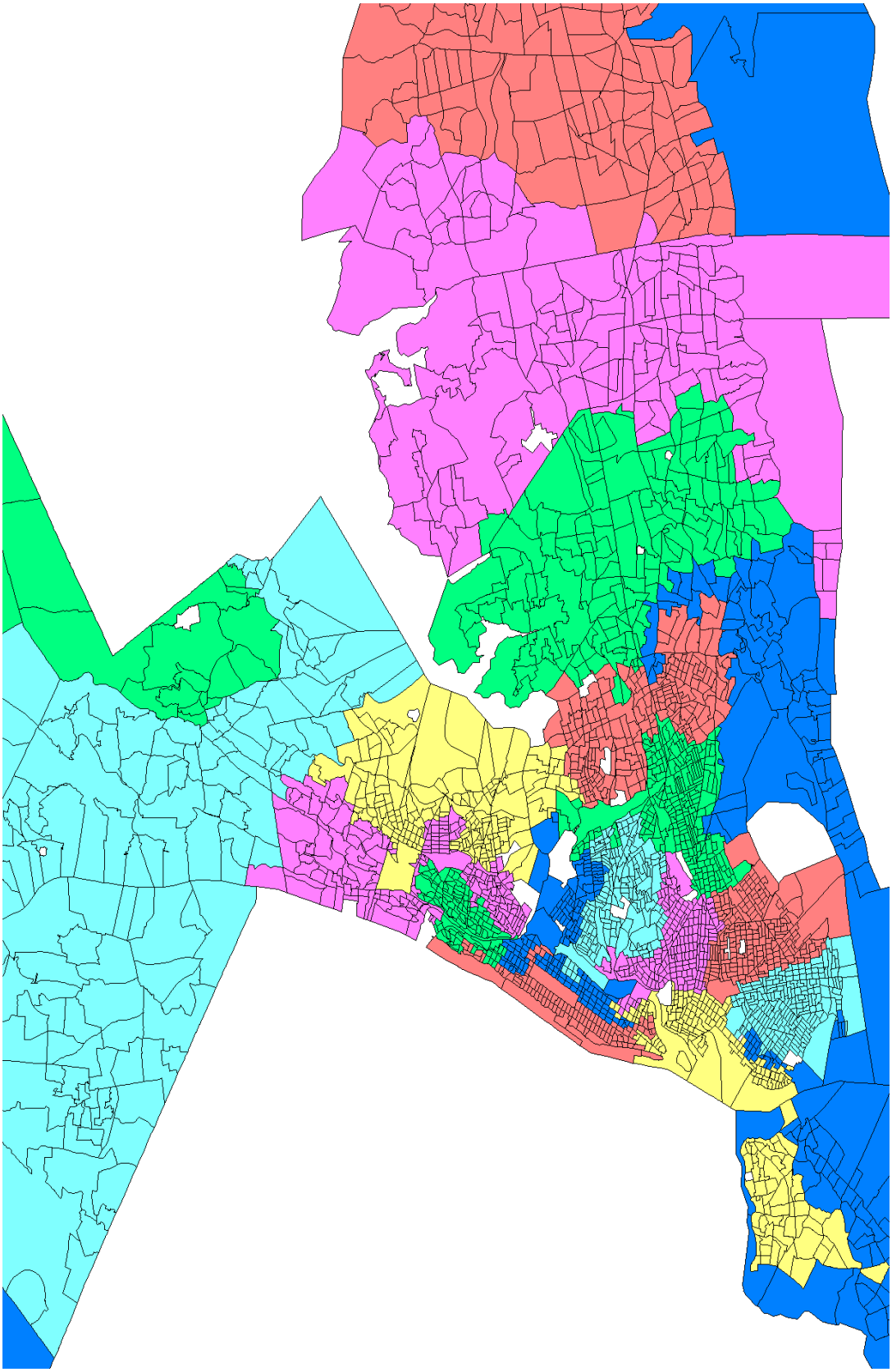


Figure 8: NYC metro-area *POM* (final optimization)

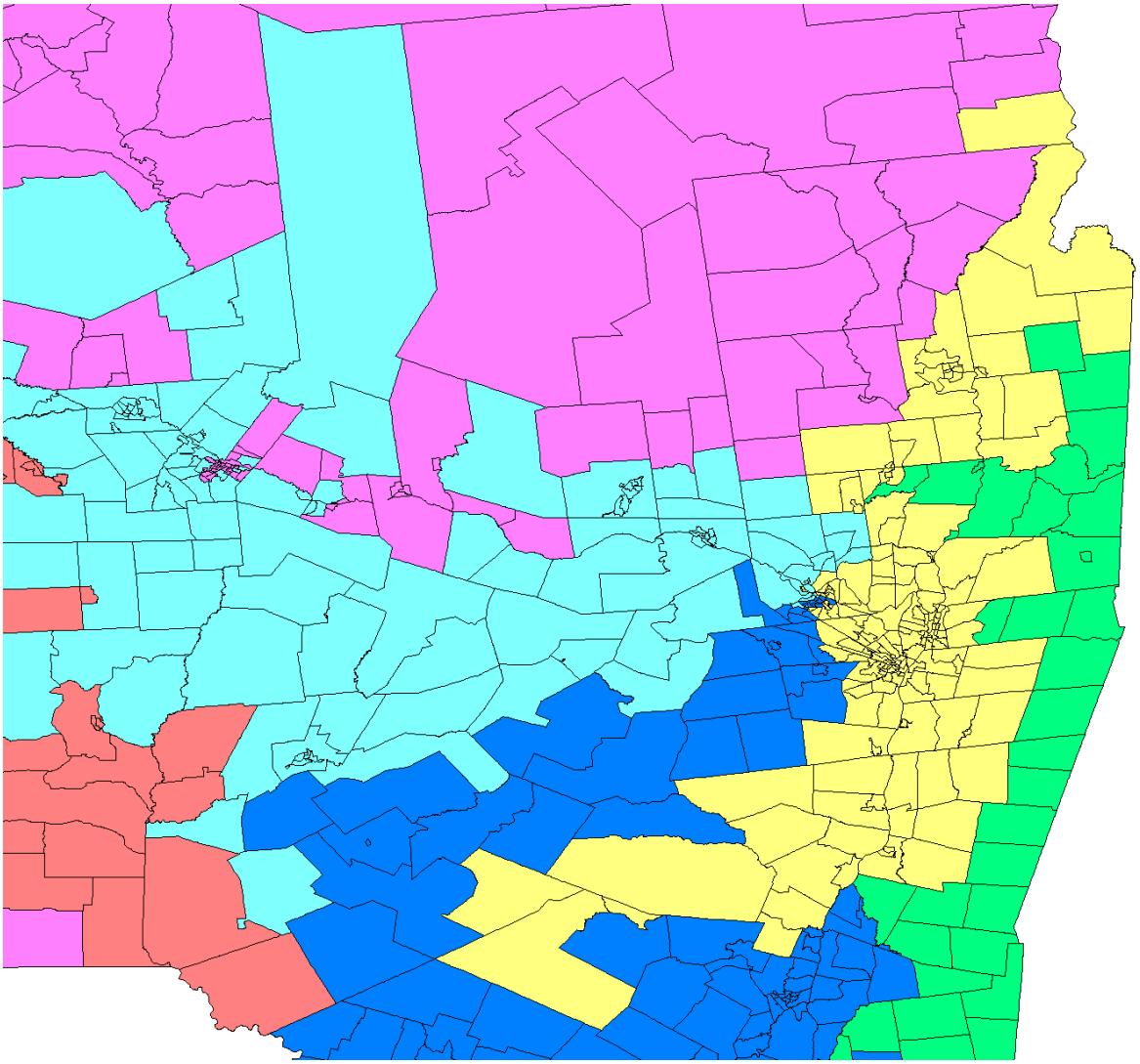


Figure 9: Close-up of the Albany area $MSGM$ (initialized districts)

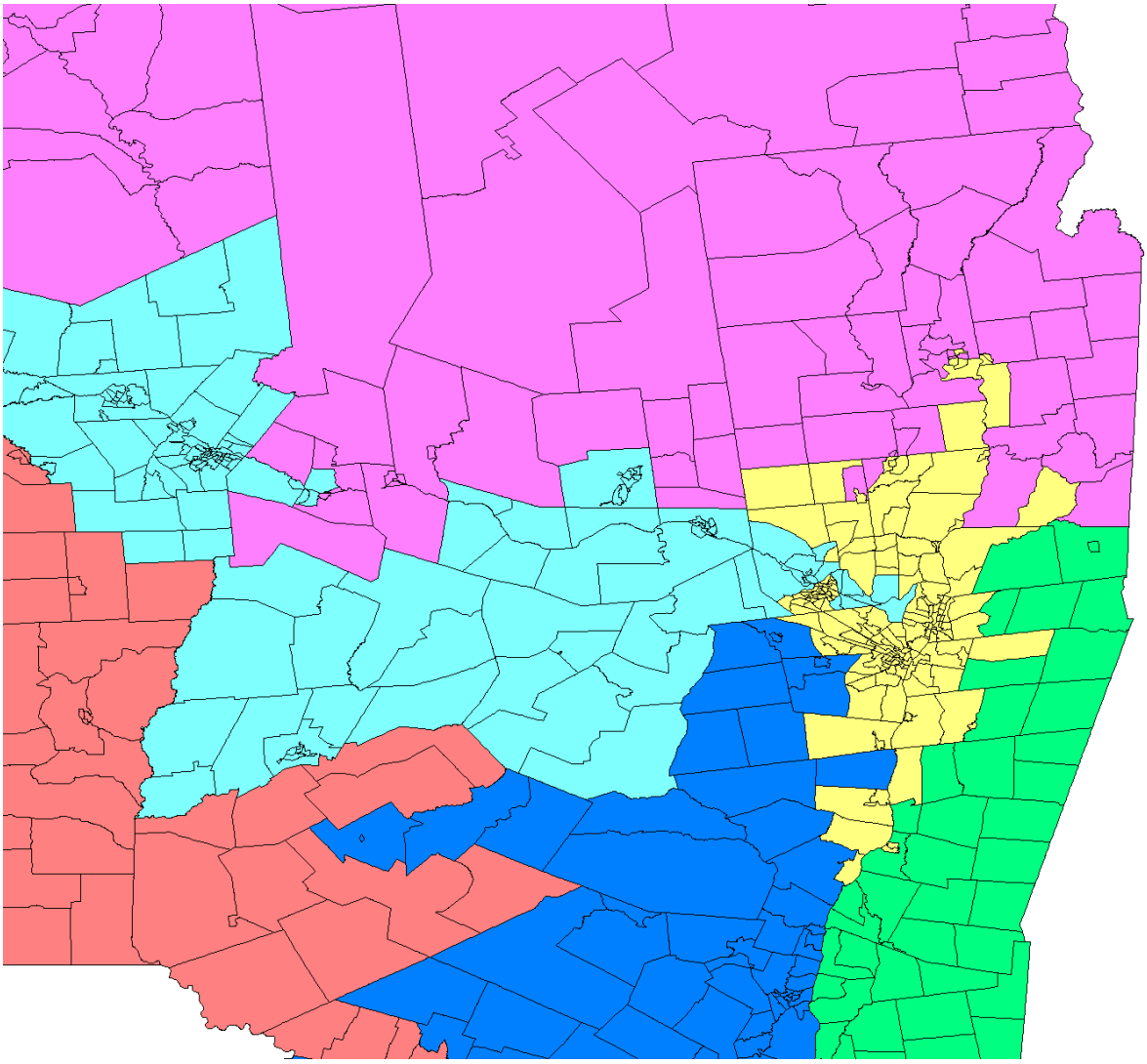


Figure 10: Close-up of the Albany area *POM* (final optimization)

B Proof of Theorem 4.1

Theorem B.1 *All connected graphs permit a paring.*

Proof We prove that any connected graph G permits a paring, by induction on the number of vertices y. We prove a stronger statement, namely that for any connected graph G with at least two vertices, there exist at least two parings. The claim clearly holds for $y = 2$.

Suppose the claim holds for $y = k$, where $k \geq 2$. Then for $y = k + 1$, suppose the claim does not hold. Then as $y \geq 3$, take any vertex v of G such that removal of v leaves G unconnected, and consider two disjoint subgraphs G_1, G_2 into which G is divided upon removal of this vertex. By the induction hypothesis, there exist vertices v_1, v_2 of G_1 such that its removal leaves G_1 connected.

I claim that removal of one of v_1, v_2 from the original graph G leaves G connected. To see this, note that neither v_1 nor v_2 is adjacent to any vertex in G_2, as G_1, G_2 have no common edges. If both v_1, v_2 are adjacent to v, then removal of v_1 leaves G connected. This is because if we let $G' = G - \{v_1\}$ and $G'_1 = G_1 - \{v_1\}$, then G' consists of $G'_1 \cup \{v\}$ and G_2, which are both connected and connected to each other, as v is necessarily connected to G_2.

This means that $G - \{v_1\}$ is connected. If one of v_1, v_2 is not adjacent to v_1, WLOG assume it is v_1. Then removing v_1 from G leaves the graph connected, as $G'_1 \cup \{v\}$ is connected, as is G_2, and they are connected to each other. Some such vertex which admits a paring also exists in G_2, yielding two vertices which permit a paring. This proves the result by induction. ∎

C Computer codes

```
1  // Tract.h - header file for a Tract
2  // a Tract has an area, a perimeter, a population, an ID, and a county.
3  // and an OGRGeometry...
4
5  #ifndef TRACT_H
6  #define TRACT_H
7
8  #include <iostream>
9  #include <vector>
10 #include <string>
11 #include <cmath>
12
13 class County;
14 class District;
15
16 using namespace std;
17
18 class Tract {
19     protected:
20         double _area;
21         double _perim;
22         int _population;
23         string _id;
24         int _county;
25         int _index;
26         OGRGeometry *_geo;
27         OGRPoint *_centroid;
28         vector<Tract *> neighbors;
29         District *_mydist;
30         County *_mycounty;
31         map<Tract *,double> shared;
32
33     public:
34         Tract(){ }
35         Tract(OGRFeature *me, int index){
36             _area = me->GetFieldAsDouble(me->GetFieldIndex("AREA"));
37             _population =
38                 me->GetFieldAsInteger(me->GetFieldIndex("TOTALPOP"));
39             _id = me->GetFieldAsString(me->GetFieldIndex("ID"));
40             string::size_type  notwhite = _id.find_first_not_of(" \t\n");
41             _id.erase(0,notwhite);
42
43             // trim trailing whitespace
44             notwhite = _id.find_last_not_of(" \t\n");
45             _id.erase(notwhite+1);
46             _county =
47                 me->GetFieldAsInteger(me->GetFieldIndex("COUNTYFP"));
48             _geo = me->StealGeometry();
49             _centroid = new OGRPoint();
50             ((OGRPolygon *)_geo)->Centroid(_centroid);
51             _index = index;
52             _perim = (((OGRPolygon *)_geo)->getExteriorRing())->get_Length();
53         }
54
55         // Setters
56         void addPerim(Tract *t,double d){
57                 shared[t] = d;
58         }
59
60         void setCounty(County *c){
61             _mycounty = c;
62         }
63
64         void setN(const vector <Tract *> &n){
65             int i;
66             for(i=0; i < n.size(); i++){
```

```
67                  if((n[i]->getPop() == 0) && (n[i]->getID() != "1491835"))
68                      continue;
69                  neighbors.push_back(n[i]);
70              }
71          }
72
73          void setDistrict(District *d){
74              _mydist = d;
75          }
76
77          // Getters
78          double getShared(Tract *t){ return shared[t]; }
79          County* getMyCounty(){ return _mycounty; }
80          int getIndex(){ return _index; }
81          vector <Tract *> getN(){ return neighbors; }
82          District *getDistrict(){ return _mydist; }
83          double getArea(){ return _area;}
84          double getPerim(){ return _perim;}
85          int getPop(){ return _population;}
86          string getID(){ return getId();}
87          string getId(){ return _id;}
88          int getCounty(){ return _county;}
89          OGRPoint* getCentroid(){ return _centroid;}
90          OGRGeometry *getGeo(){
91              return _geo;
92          }
93
94          // Neat stuff I can do with Tracts
95          double bcMetric(Tract *t){
96              return distC(t)/min(getArea(),(t->getArea()));
97          }
98
99          double getPopDen(){
100             return getPop()/getArea();
101         }
102
103         bool bordersp(Tract *t){
104             OGRGeometry *g = t->getGeo();
105             return _geo->Touches(g);
106         }
107
108         double distBetweenTracts(Tract *t){
109             OGRGeometry *g = t->getGeo();
110             return _geo->Distance(g);
111         }
112
113         double dist(OGRPoint *oc){
114             double xdiff = _centroid->getX() - oc->getX();
115             double ydiff = _centroid->getY() - oc->getY();
116             return sqrt(xdiff*xdiff + ydiff*ydiff);
117         }
118
119         double distC(Tract *t){
120             OGRPoint *oc = t->getCentroid();
121             double xdiff = _centroid->getX() - oc->getX();
122             double ydiff = _centroid->getY() - oc->getY();
123             return sqrt(xdiff*xdiff + ydiff*ydiff);
124         }
125
126         bool onPerimeter(){
127             // returns true iff exists a neighboring tract with a
128             // different district assignment
129             vector <Tract *>::iterator iter;
130             for(iter = neighbors.begin(); iter != neighbors.end(); iter++){
131                 if((*iter)->getDistrict() != _mydist){
132                     return true;
133                 }
134             }
```

```
135             return false;
136         }
137
138         vector <District *> getNColors(){
139             // returns list of districts touching this one...
140             int i;
141             vector<Tract *> n = getN();
142             map<District *,bool> seenit;
143             vector<District *> retval;
144
145             for(i=0; i < n.size(); i++){
146                 if((n[i]->getDistrict() != getDistrict()) &&
147                         !seenit[n[i]->getDistrict()]){
148                     retval.push_back(n[i]->getDistrict());
149                     seenit[n[i]->getDistrict()] = true;
150                 }
151             }
152             return retval;
153         }
154
155 };
156
157 #endif
```

```
1
2  // Fnode.h - defines a Fronteir node structure, consisting of a Tract
3  // and the District to change that Tract to.
4
5  #include <iostream>
6  #include "Tract.h"
7  #include "District.h"
8
9  class Fnode {
10         private:
11                 Tract *_t;
12                 District *_d;
13                 double _score;
14
15         public:
16                 Fnode() { }
17                 Fnode(Tract *t, District *d){
18                         _t = t;
19                         _d = d;
20                 }
21
22                 void setScore(double score){
23                         _score = score;
24                 }
25
26                 double getScore(){
27                         return _score;
28                 }
29
30                 Tract *getTract(){
31                         return _t;
32                 }
33
34                 District *getDistrict(){
35                         return _d;
36                 }
37 };
```

```
1  // County.h - header file for a County
2  // a County consists of a list of pointers to tracts.
3
4  #ifndef COUNTY_H
5  #define COUNTY_H
6
```

```
 7 #include <iostream>
 8 #include <vector>
 9 #include <map>
10 #include "Tract.h"
11
12 class District;
13
14 using namespace std;
15 // NDIST?
16 extern District* BLANKDIST;
17
18 class County {
19     protected:
20         vector<Tract *> myTracts;
21         double area;
22         int population;
23
24     public:
25         County(){
26             population = 0;
27             area = 0;
28         }
29         void addToCounty(Tract *t){
30             myTracts.push_back(t);
31             area += t->getArea();
32             population += t->getPop();
33         }
34
35         void printCounty(){
36             //map<District *,int> p;
37             map<District *,double> a;
38
39             map<int,string> cnames;
40             cnames[3] = "Allegeny";
41             cnames[13] = "Chautauqua";
42             cnames[9] = "Cattaraugus";
43             cnames[29] = "Erie";
44             cnames[63] = "Niagara";
45             cnames[73] = "Orleans";
46             cnames[37] = "Genesee";
47             cnames[121] = "Wyoming";
48             cnames[55] = "Monroe";
49             cnames[51] = "Livingston";
50             cnames[117] = "Wayne";
51             cnames[101] = "Steuben";
52             cnames[69] = "Ontario";
53             cnames[123] = "Yates";
54             cnames[11] = "Cayuga";
55             cnames[97] = "Schuyler";
56             cnames[99] = "Seneca";
57             cnames[15] = "Chemung";
58             cnames[33] = "Franklin";
59             cnames[109] = "Tompkins";
60             cnames[107] = "Tioga";
61             cnames[23] = "Cortland";
62             cnames[75] = "Oswego";
63             cnames[45] = "Jefferson";
64             cnames[89] = "St. Lawrence";
65             cnames[49] = "Lewis";
66             cnames[67] = "Onondaga";
67             cnames[7] = "Broome";
68             cnames[17] = "Chenango";
69             cnames[43] = "Herkimer";
70             cnames[41] = "Hamilton";
71             cnames[31] = "Essex";
72             cnames[113] = "Warren";
73             cnames[19] = "Clinton";
74             cnames[115] = "Washington";
```

```
75              cnames[83] = "Rensselaer";
76              cnames[21] = "Columbia";
77              cnames[27] = "Dutchess";
78              cnames[91] = "Saratoga";
79              cnames[35] = "Fulton";
80              cnames[93] = "Schenectady";
81              cnames[57] = "Montgomery";
82              cnames[25] = "Delaware";
83              cnames[77] = "Otsego";
84              cnames[65] = "Oneida";
85              cnames[53] = "Madison";
86              cnames[21] = "Columbia";
87              cnames[27] = "Dutchess";
88              cnames[79] = "Putnam";
89              cnames[119] = "Westchester";
90              cnames[105] = "Sullivan";
91              cnames[71] = "Orange";
92              cnames[111] = "Ulster";
93              cnames[39] = "Greene";
94              cnames[95] = "Schoharie";
95              cnames[1] = "Albany";
96              cnames[87] = "Rockland";
97              cnames[103] = "Suffolk";
98              cnames[59] = "Nassau";
99              cnames[81] = "Queens";
100             cnames[85] = "Richmond";
101             cnames[47] = "Kings";
102             cnames[5] = "Bronx";
103             cnames[61] = "New York";
104
105             int i;
106             for(i=0; i < myTracts.size(); i++){
107                 //p[myTracts[i]->getDistrict()] += myTracts[i]->getPop();
108                 a[myTracts[i]->getDistrict()] += myTracts[i]->getArea();
109             }
110
111             double x;
112             double largest = -1;
113             //map<District *,int>::iterator piter;
114             map<District *,double>::iterator aiter;
115             cout << cnames[myTracts.front()->getCounty()] << " ";
116             for(aiter = a.begin(); aiter != a.end(); aiter++){
117                 x = (double)(aiter->second)/(double)getArea();
118                 if(x > largest){
119                         largest = x;
120                 }
121             }
122             cout << largest << " " << a.size() << endl;
123         }
124
125         vector<Tract *> getTractList(){
126             return myTracts;
127         }
128
129         int getPop(){
130             return population;
131         }
132
133         double getArea(){
134             return area;
135         }
136
137         double getValue(){
138             double scale = 1e7;
139
140             map<District *,int> p;
141             //map<District *,double> a;
142
```

```
143             double a = 1.0 * scale;
144
145             int i;
146             for(i=0; i < myTracts.size(); i++){
147                 if(myTracts[i]->getDistrict() != BLANKDIST){
148                     p[myTracts[i]->getDistrict()] += myTracts[i]->getPop();
149                 }
150                 //a[myTracts[i]] += myTracts[i]->getArea();
151             }
152
153             double returnval=0;
154             double x;
155             map<District *,int>::iterator piter;
156             //map<District *,double>::iterator aiter;
157
158             for(piter = p.begin(); piter != p.end(); piter++){
159                 x = (double)(piter->second)/(double)population;
160                 returnval += a*x*x;
161             }
162
163             return returnval;
164         }
165
166 };
167
168 #endif
```

```
1
2  // District.h - header file for a District
3  // a District consists of a list of tracts, area, perimeter, and
4  // population.
5
6  #ifndef DISTRICT_H
7  #define DISTRICT_H
8
9  #include <iostream>
10 #include <list>
11 #include <map>
12 #include <vector>
13 #include "Tract.h"
14 #include "County.h"
15 #include <sstream>
16
17
18 using namespace std;
19 extern District *BLANKDIST;
20 extern const double AVGPEOPLE;
21 extern bool comp_func(Tract *lhs, Tract *rhs);
22 extern bool eq_func(Tract *lhs, Tract *rhs);
23
24 class District {
25     protected:
26         list<Tract *> myTracts;
27         double _area;
28         double _perimeter;
29         int _population;
30         int _numtracts;
31
32     public:
33         District() {
34             _area = 0;
35             _population = 0;
36             _numtracts = 0;
37         }
38
39         void removeFromDistrict(Tract *t){
40             myTracts.remove(t);
41             _numtracts--;
42             _area =  _area - t->getArea();
```

```
43              _population = _population - t->getPop();
44          }
45
46          void addToDistrict(Tract *t){
47              if((t->getPop() == 0) && (t->getID() != "1491835"))
48                  return;
49              myTracts.push_front(t);
50              _numtracts++;
51              _area += t->getArea();
52              //_perimeter += t->getPerimeter();
53              // would need to do pairwise elimination on borders...
54              _population += t->getPop();
55          }
56
57          double getArea(){
58              return _area;
59          }
60
61          /*
62             double getPerimeter(){
63             return _perimeter;
64             }*/
65
66          double getIsoPerim(){
67              double scale = .001;
68              OGRGeometry *uni;
69              list<Tract *>::iterator liter;
70              list<Tract *> l = getPerimeter();
71              vector<Tract *> n;
72              double p=0;
73              int i;
74              double count;
75              //uni = ((myTracts.front())->getGeo())->clone();
76
77              for(liter = l.begin(); liter != l.end(); liter++){
78                  count = 0;
79                  n = (*liter)->getN();
80                  for(i=0; i < n.size(); i++){
81                      if(n[i]->getDistrict() != this ){
82                          count++;
83                      }
84                  }
85                  p += ((*liter)->getPerim())*(count/(double)n.size());
86              }
87              /*
88              for(liter = myTracts.begin(); liter != myTracts.end();
89                      liter++){
90                  n = (*liter)->getN();
91                  p = p + (*liter)->getPerim();
92                  for(i=0; i < n.size(); i++){
93                      if(n[i]->getDistrict() == this){
94                          p += n[i]->getPerim() - (*liter)->getShared(n[i]);
95                      }
96                  }
97              }*/
98
99              //double a = ((OGRPolygon *)uni)->get_Area();
100             double a = getArea();
101             //OGRLinearRing *perim = ((OGRPolygon*)uni)->getExteriorRing();
102             //double p = perim->get_Length();
103             //cout << "Area " << a << " Perimeter " << p << endl;
104             //delete uni;
105
106             return scale*a/(p*p);
107         }
108
109
110         int getPop(){
```

```
111             return _population;
112         }
113 
114         int getNumTracts(){
115             return _numtracts;
116         }
117 
118         list<Tract *> getTractList(){
119             return myTracts;
120         }
121 
122         double score(){
123             return newcountyScore() + compactScore() + varScore() + countyScore();
124         }
125 
126         double newcountyScore(){
127             double M = 0.;
128             list<Tract *>::iterator liter;
129             map<County *,double> pz;
130             County *c;
131             double frac;
132             map<County *,double>::iterator miter;
133             double retval=0;
134 
135             for(liter = myTracts.begin(); liter != myTracts.end();
136                     liter++){
137                 c = (*liter)->getMyCounty();
138                 pz[c] += (*liter)->getArea();
139             }
140 
141             return M*pz.size();
142         }
143 
144         double countyScore(){
145             double M = 1.;
146             list<Tract *>::iterator liter;
147             map<County *,double> pz;
148             map<int,string> cnames;
149             County *c;
150             double frac;
151             map<County *,double>::iterator miter;
152             double retval=0;
153 // Initialize County Names
154             cnames[3] = "Allegeny";
155             cnames[13] = "Chautauqua";
156             cnames[9] = "Cattaraugus";
157             cnames[29] = "Erie";
158             cnames[63] = "Niagara";
159             cnames[73] = "Orleans";
160             cnames[37] = "Genesee";
161             cnames[121] = "Wyoming";
162             cnames[55] = "Monroe";
163             cnames[51] = "Livingston";
164             cnames[117] = "Wayne";
165             cnames[101] = "Steuben";
166             cnames[69] = "Ontario";
167             cnames[123] = "Yates";
168             cnames[11] = "Cayuga";
169             cnames[97] = "Schuyler";
170             cnames[99] = "Seneca";
171             cnames[15] = "Chemung";
172             cnames[33] = "Franklin";
173             cnames[109] = "Tompkins";
174             cnames[107] = "Tioga";
175             cnames[23] = "Cortland";
176             cnames[75] = "Oswego";
177             cnames[45] = "Jefferson";
178             cnames[89] = "St. Lawrence";
```

```
179             cnames[49] = "Lewis";
180             cnames[67] = "Onondaga";
181             cnames[7] = "Broome";
182             cnames[17] = "Chenango";
183             cnames[43] = "Herkimer";
184             cnames[41] = "Hamilton";
185             cnames[31] = "Essex";
186             cnames[113] = "Warren";
187             cnames[19] = "Clinton";
188             cnames[115] = "Washington";
189             cnames[83] = "Rensselaer";
190             cnames[21] = "Columbia";
191             cnames[27] = "Dutchess";
192             cnames[91] = "Saratoga";
193             cnames[35] = "Fulton";
194             cnames[93] = "Schenectady";
195             cnames[57] = "Montgomery";
196             cnames[25] = "Delaware";
197             cnames[77] = "Otsego";
198             cnames[65] = "Oneida";
199             cnames[53] = "Madison";
200             cnames[21] = "Columbia";
201             cnames[27] = "Dutchess";
202             cnames[79] = "Putnam";
203             cnames[119] = "Westchester";
204             cnames[105] = "Sullivan";
205             cnames[71] = "Orange";
206             cnames[111] = "Ulster";
207             cnames[39] = "Greene";
208             cnames[95] = "Schoharie";
209             cnames[1] = "Albany";
210             cnames[87] = "Rockland";
211             cnames[103] = "Suffolk";
212             cnames[59] = "Nassau";
213             cnames[81] = "Queens";
214             cnames[85] = "Richmond";
215             cnames[47] = "Kings";
216             cnames[5] = "Bronx";
217             cnames[61] = "New␣York";
218
219             for(liter = myTracts.begin(); liter != myTracts.end();
220                     liter++){
221                 c = (*liter)->getMyCounty();
222                 pz[c] += (*liter)->getArea();
223             }
224
225             for(miter = pz.begin(); miter != pz.end(); miter++){
226                 frac =
227                     (double)(miter->second)/(double)((miter->first)->getArea());
228                 cout <<
229                     cnames[((miter->first)->getTractList()).front()->getCounty()]
230                     << "␣" << frac << "␣";
231                 retval += frac*frac;
232             }
233             cout << endl;
234             return M*(retval);
235         }
236
237         OGRPoint *centerOfMass(){
238             list<Tract *>::iterator liter;
239             double x=0,y=0;
240
241             for(liter = myTracts.begin(); liter != myTracts.end();
242                     liter++){
243                 x += ((*liter)->getPop())*((*liter)->getCentroid())->getX();
244                 y +=
245                     ((*liter)->getPop())*((*liter)->getCentroid())->getY();
246             }
```

```
247             x = x/getPop();
248             y = y/getPop();
249 
250             OGRPoint *retval = new OGRPoint();
251             retval->setX(x);
252             retval->setY(y);
253             return retval;
254         }
255         double bcBB(){
256             list<Tract *>::iterator perim;
257             list<Tract *> p = myTracts;
258             double minX = 999999999999.;
259             double minY = 999999999999.;
260             double maxX = -999999999999.;
261             double maxY = -999999999999.;
262             double curX;
263             double curY;
264             OGRPoint *pt;
265 
266             for(perim = p.begin(); perim != p.end(); perim++){
267                 pt = (*perim)->getCentroid();
268                 curX = pt->getX();
269                 curY = pt->getY();
270                 if(curX < minX){
271                     minX = curX;
272                 }
273                 if(curY < minY){
274                     minY = curY;
275                 }
276                 if(curX > maxX){
277                     maxX = curX;
278                 }
279                 if(curY > maxY){
280                     maxY = curY;
281                 }
282             }
283 
284             return 4.*pow(maxX-minX+(maxY-minY),2);
285         }
286 
287         vector<District *> whatBordersMe(){
288             map<District *,bool>seenit;
289             list<Tract *>::iterator perim;
290             list<Tract *> p = getFrontier();
291             vector<District *> retval;
292             for(perim = p.begin(); perim != p.end(); perim++){
293                 if(!seenit[(*perim)->getDistrict()]){
294                     seenit[(*perim)->getDistrict()] = true;
295                     retval.push_back((*perim)->getDistrict());
296                 }
297             }
298 
299             return retval;
300         }
301 
302         vector<Tract *> sharesBorder(District *d){
303             list<Tract *>::iterator perim;
304             list<Tract *> p = getFrontier();
305             vector<Tract *> retval;
306             for(perim = p.begin(); perim != p.end(); perim++){
307                 if((*perim)->getDistrict() == d){
308                     retval.push_back(*perim);
309                 }
310             }
311 
312             return retval;
313         }
314 
```

```
315
316         double compactScore(){
317             //return getIsoPerim();
318             //double M = .1;
319             //return M*getArea()/bcBB();
320             /*
321             double M = -10000;
322             list<Tract *>::iterator perim;
323             list<Tract *> p = myTracts;
324             double avgDist = 0;
325             OGRPoint *c = centerOfMass();
326             for(perim = p.begin(); perim != p.end(); perim++){
327                 avgDist += (*perim)->dist(c);
328             }
329             avgDist = avgDist/p.size();
330             double retval = 0;
331             for(perim = p.begin(); perim != p.end(); perim++){
332                 retval = pow((1-(*perim)->dist(c)/avgDist),2.);
333             }
334             delete c;
335             return M*retval/(p.size()-1);
336             */
337
338             double M = 30;
339             list<Tract *>::iterator liter;
340             vector<Tract *> n;
341             int i;
342             double count = 0;
343             for(liter = myTracts.begin(); liter != myTracts.end();
344                     liter++){
345                 n = (*liter)->getN();
346                 for(i=0; i < n.size(); i++){
347                     if(n[i]->getDistrict() == this){
348                         count++;
349                     }
350                 }
351             }
352             return count * M /
353                 (((double)myTracts.size())*((double)myTracts.size() - 1));
354             //
355            /*
356             double M = 10.;
357             list<Tract *> p = getPerimeter();
358             double b = (double)p.size();
359             int nt = getNumTracts();
360             return M*((double)nt)/pow(b+4.,2.);*/
361         }
362
363         inline double varScore(){
364             double M = -1000.;
365             return M*(getPop() -
366                     AVGPEOPLE)*(1./getPop())*(getPop()-AVGPEOPLE)*(1./getPop());
367         }
368
369         map<Tract *,bool> visited;
370
371         bool isContiguous(){
372             if(this == BLANKDIST){
373                 return true;
374             }
375             list <Tract *>::iterator liter;
376             visited.clear();
377             dfs(getTractList().front());
378             bool visitedall = true;
379             for(liter = myTracts.begin(); liter != myTracts.end();
380                     liter++){
381                 if(visited[(*liter)] == false){
382                     visitedall = false;
```

```
383                     break;
384                 }
385             }
386 
387             return visitedall;
388         }
389 
390         void dfs(Tract *t){
391             visited[t] = true;
392             vector <Tract *> n = t->getN();
393             int i;
394             for(i=0; i < n.size(); i++){
395                 if((this == n[i]->getDistrict()) && (!visited[n[i]])){
396                     dfs(n[i]);
397                 }
398             }
399         }
400 
401         double getValue(){
402             if(this == BLANKDIST){
403                 return 0;
404             }
405             double M=10000;
406             double p = (double)getPop();
407             if(p < AVGPEOPLE){
408                 return M*sqrt(p/AVGPEOPLE);
409             }
410             return M-4*M*((p-AVGPEOPLE)/p)*((p-AVGPEOPLE)/p);
411         }
412 
413         // perimeter -> set of nodes that are in this and border
414         // something not in this
415         list<Tract *> getPerimeter(){
416             // go through all the Tracts...
417             list<Tract *>::iterator liter;
418             list<Tract *> returnval;
419             for(liter = myTracts.begin(); liter != myTracts.end();
420                     liter++){
421                 if((*liter)->onPerimeter()){
422                     returnval.push_front(*liter);
423                 }
424             }
425             return returnval;
426         }
427 
428 
429         // frontier -> set of nodes that border this
430         list<Tract *> getFrontier(){
431             // go thru all the vectors
432             // add to master list only if it's not == this
433             list<Tract *>::iterator liter;
434             list<Tract *> returnval;
435             map<Tract *,bool> seenit;
436 
437             vector<Tract *> v;
438             int i;
439             for(liter = myTracts.begin(); liter != myTracts.end();
440                     liter++){
441                 v = (*liter)->getN();
442                 for(i=0; i < v.size(); i++){
443                     if((this != v[i]->getDistrict()) && !seenit[v[i]]){
444                         returnval.push_front(v[i]);
445                         seenit[v[i]] = true;
446                     }
447                 }
448             }
449 
450             //returnval.sort();
```

```
451             /*
452             Tract *prev;
453             if(returnval.size() > 1){
454                 prev = returnval.front();
455                 for(liter = ((returnval.begin())++); liter != returnval.end();
456                         liter++){
457                         if(prev == (*liter)){
458                             returnval.remove(prev);
459                             addin.push_back(prev);
460                         }
461                         prev = *liter;
462                 }
463             }
464             for(i=0; i < addin.size(); i++){
465                 returnval.push_front(addin[i]);
466             }*/
467             //returnval.unique();
468             return returnval;
469         }
470 /*
471         bool minmex(const Tract* a,const Tract *b){
472             return ((a.getCentroid())->getX() <
473                     (b.getCentroid())->getX());
474         }
475 
476         double *getMinMaxX(){
477             double returnval[2];
478             myTracts.sort(minmex);
479             returnval[0] = (myTracts.front()->getCentroid())->getX();
480             returnval[1] = (myTracts.back()->getCentroid())->getX();
481             return returnval;
482         }
483 
484         bool minmey(const Tract* a, const Tract *b){
485             return ((a.getCentroid())->getY() <
486                     (b.getCentroid())->getY());
487         }
488 
489         double *getMinMaxY(){
490             double returnval[2];
491             myTracts.sort(minmey);
492             returnval[0] = (myTracts.front()->getCentroid())->getY();
493             returnval[1] = (myTracts.back()->getCentroid())->getY();
494             return returnval;
495         }
496 
497         list<Tract *> cleavelessthanx(double target){
498             list<Tract *> returnval;
499             list<Tract *>::iterator iter;
500             for(iter=myTracts.start();iter != myTracks.end(); iter++){
501                 if(((*iter)->getCentroid())->getX() < target){
502                     myTracts.remove(*iter);
503                     returnval.push_back(*iter);
504                 }
505             }
506         }*/
507 };
508 
509 #endif
```

```
1 
2 // Allocation.h - header file for an Allocation
3 // an Allocation consists of an array of districts (29) and a heuristic
4 // value.
5 
6 #ifndef ALLOCATION_H
7 #define ALLOCATION_H
8 
9 #include <iostream>
```

```
10 #include <cmath>
11 #include "District.h"
12
13 using namespace std;
14
15 class Allocation {
16     protected:
17         District* d[29];
18
19     public:
20         Allocation(){ }
21         Allocation(District **ds){
22             int i;
23             for(i=0; i < 29; i++){
24                 d[i] = ds[i];
25             }
26         }
27
28         District **getDistricts(){
29             return d;
30         }
31 };
32
33 #endif
```

```
 1
 2
 3 #include "ogrsf_frmts.h"
 4 #include <iostream>
 5 #include <fstream>
 6 #include <iomanip>
 7 #include <string>
 8 #include <map>
 9 #include "Tract.h"
10 #include "County.h"
11 #include "District.h"
12 #include "Allocation.h"
13 //#include "rng.h"
14 #include <sstream>
15 #include <cstdlib>
16 #include <ctime>
17 #include <vector>
18 #include "Fnode.h"
19 #include <algorithm>
20
21 const int NTRACT = 4907;
22 const int NDIST = 29;
23 const double AVGPEOPLE = 18976457./(float)NDIST;
24 const int NCOUNTY = 62;
25 //const int NLEVELS = 20;
26 District *BLANKDIST;
27 const bool PRINTHEU = false;
28
29 using namespace std;
30
31 void plotAllocation(Allocation *a,string fname);
32 District **getNeighbor(District **d, Tract** allTracts, double
33         **distmat);
34 void moveTract(Tract *t, District *newd);
35 double getBadness(District **d,double **distmat);
36 void clarify(Tract **allTracts);
37 void addneighrecur(Tract *t,District *changeto,District *background,int
38         levels);
39 double generateScore(District **d, County **allCounties);
40 vector <Fnode *> unionFrontier(District **d);
41 double getBC(vector<Tract *> startingpoints,Tract *t);
42
43 bool compf(Fnode *lhs, Fnode *rhs){
44     // greater than, not less than, b/c we want to sort descending
```

```
45      return lhs->getScore() > rhs->getScore();
46  }
47
48  bool eqf(Fnode *lhs, Fnode *rhs){
49      return lhs->getScore() == rhs->getScore();
50  }
51
52  bool eq_func(Tract *lhs, Tract *rhs){
53      return lhs == rhs;
54  }
55  bool compbefore(Fnode *lhs, Fnode *rhs){
56      if(rhs->getTract() >= lhs->getTract()){
57          return true;
58      } else if(rhs->getTract() == lhs->getTract()){
59          if(rhs->getDistrict() >= lhs->getDistrict()){
60              return true;
61          }
62      }
63      return false;
64  }
65
66  bool eqbefore(Fnode *lhs, Fnode *rhs){
67      return((rhs->getTract() == lhs->getTract()) &&
68              (rhs->getDistrict() == lhs->getDistrict()));
69  }
70
71  bool comp_func(Tract *lhs, Tract *rhs){
72      return lhs < rhs;
73  }
74
75  string inttostring(const int i){
76      ostringstream stream;
77      stream << i;
78      return stream.str();
79  }
80
81  double randdub(){
82      return rand()/(double)RAND_MAX;
83  }
84  //returns between lo and hi inclusive
85  int randint(int low, int high){
86      return(low+(int)floor(randdub()*(high-low+1)));
87  }
88
89  vector <Tract *>copyvec(const vector<Tract *> &in){
90      int i;
91      vector <Tract *> returnval;
92      for(i=0;i<in.size();i++){
93          returnval[i] = in[i];
94      }
95  }
96
97  int main(int argc, char * const argv[]) {
98      srand((unsigned)time(NULL));
99
100     OGRRegisterAll();
101
102     OGRDataSource *myfile;
103
104     myfile = OGRSFDriverRegistrar::Open("./polygons/", FALSE);
105     if(myfile == NULL){
106         cerr << "Can't open file" << endl;
107         return 1;
108     }
109     cout << "Opened file appropriately!" << endl;
110     cout << "File has " << myfile->GetLayerCount() << " layers" << endl;
111
112     OGRLayer *layer = myfile->GetLayer(0);
```

```
113     if(!layer){
114         cerr << "Cannot open layer" << endl;
115         return 1;
116     }
117 
118     cout << "Layer has " << layer->GetFeatureCount() << " features" <<
119         endl;
120     int numtracts = layer->GetFeatureCount();
121     int i,j;
122     OGRFeature *feat;
123     int populationindex;
124     int totalpop = 0;
125     map<string,int> IDtoIref;
126     map<int,int> CkeytoRkey; // county key in file to our real keys.
127     Tract *allTracts[NTRACT];
128     bool **bmat = new bool*[NTRACT];
129     double **distmat = new double*[NTRACT];
130 
131     double pdscore,pcscore,fdscore,fcscore;
132     Allocation *a;
133     County **allCounties = new County*[NCOUNTY];
134     for(i=0; i < NCOUNTY; i++){
135         allCounties[i] = new County();
136     }
137 
138     int cindex=-1;
139     for(i=0; i < numtracts; i++){
140         feat = layer->GetNextFeature();
141         if(!feat){
142             cerr << "Could not read feature, exiting!" << endl;
143             return 1;
144         }
145         allTracts[i] = new Tract(feat,i);
146         IDtoIref[allTracts[i]->getID()] = i;
147         // Link to counties...
148         if(CkeytoRkey.count(allTracts[i]->getCounty()) == 0){
149             cindex++;
150             CkeytoRkey[allTracts[i]->getCounty()] = cindex;
151         }
152         allCounties[CkeytoRkey[allTracts[i]->getCounty()]]->addToCounty(allTracts[i]);
153 
154         delete feat;
155         feat = NULL;
156     }
157 
158     cout << "beginning to read border file..." << endl;
159     ifstream bo;
160     bo.open("border.txt");
161     for(i=0; i < NTRACT; i++){
162         bmat[i] = new bool[NTRACT];
163         for(j=0; j < NTRACT; j++){
164             bo >> bmat[i][j];
165         }
166     }
167     bo.close();
168     cout << "finished reading border file" << endl;
169 
170     vector <Tract *> n;
171     for(i=0; i < NTRACT; i++){
172         for(j=0; j < NTRACT; j++){
173             if(bmat[i][j]){
174                 n.push_back(allTracts[j]);
175             }
176         }
177         allTracts[i]->setN(n);
178         n.clear();
179     }
180 
```

```
181     cout << "beginning calculating centroid distances" << endl;
182     for(i=0; i < NTRACT; i++){
183         distmat[i] = new double[NTRACT];
184         for(j=0; j < NTRACT; j++){
185             if(j < i){
186                 distmat[i][j] = distmat[j][i];
187             } else {
188                 distmat[i][j] = allTracts[i]->distC(allTracts[j]);
189             }
190         }
191     }
192     cout << "finished calculating centroid distances" << endl;
193
194     District *d[NDIST+1]; // d[NDIST] = blank canvas....
195     for(i=0; i < NDIST+1; i++){
196         d[i] = new District();
197     }
198
199     BLANKDIST = d[NDIST];
200
201     // initially we paint everything NDIST...
202     for(i=0; i < NTRACT; i++){
203         allTracts[i]->setDistrict(BLANKDIST);
204         BLANKDIST->addToDistrict(allTracts[i]);
205     }
206
207
208     string spoint = "     3483864"; // remember the spaces!
209     int iref = IDtoIref[spoint];
210
211     if(!allTracts[iref]){
212         cerr << "Could not find starting node, exiting!" << endl;
213         return 1;
214     }
215
216     // color it!
217     District *curd;
218     //curd = d[0];
219     //moveTract(allTracts[iref],curd);
220     // in each step, get list of possible frontier nodes.
221     // find the value of adding each node.
222     // add the one with highest value only if the new value is increased
223     District *checkme;
224     list <Tract *> f;
225     list <Tract *>::iterator liter;
226     double hiscore;
227     Tract *addme;
228     double curval,tmpscore;
229     bool done;
230     hiscore = -999999999;
231     addme = NULL;
232     Tract *abba;
233     vector <Tract *> startingpoints;
234     double maxdist;
235     Tract *thevest; // "vest is best!"
236     abba = allTracts[iref];
237     /* distance maximin
238     startingpoints.push_back(abba);
239     for(i=1; i < NDIST; i++){
240         maxdist = -1.;
241         for(j=0; j < NTRACT; j++){
242             tmpscore=getBC(startingpoints,allTracts[j]);
243             if(tmpscore > maxdist){
244                 maxdist = tmpscore;
245                 thevest = allTracts[j];
246             }
247         }
248         startingpoints.push_back(thevest);
```

```
249     }
250     for(i=0; i < NDIST; i++){
251         abba = startingpoints[i];
252         moveTract(abba,d[i]);
253     }*/
254     bool flag;
255
256     cout << "Allocating initial random districts" << endl;
257     for(i=0; i <NDIST; i++){
258         flag = false;
259         do {
260             j = randint(0,NTRACT-1);
261             abba = allTracts[j];
262             if(randdub() < (double)abba->getPop()/25000.)
263                 flag = true;
264         } while(abba->getDistrict() != BLANKDIST || !flag);
265         moveTract(abba,d[i]);
266     }
267     cout << "Done random allocation" << endl;
268     a = new Allocation(d);
269     plotAllocation(a, "initial");
270     vector<Fnode *> curfr;
271     Fnode *best;
272     County *iq;
273
274     while((BLANKDIST->getTractList()).size() > 0){
275         curfr = unionFrontier(d);
276         //sort(curfr.begin(),curfr.end(),compbefore);
277         //curfr.erase(unique(curfr.begin(),curfr.end(),eqbefore),curfr.end());
278         cout << "Current size is: " <<
279             (BLANKDIST->getTractList()).size() <<
280             " Frontier: " << curfr.size() << endl;
281         for(i=0; i < curfr.size(); i++){
282             pdscore = (curfr[i]->getDistrict())->getValue();
283             iq =
284                 allCounties[CkeytoRkey[(curfr[i]->getTract())->getCounty()]];
285             pcscore = iq->getValue();
286             moveTract(curfr[i]->getTract(),curfr[i]->getDistrict());
287             fdscore = (curfr[i]->getDistrict())->getValue();
288             fcscore = iq->getValue();
289             //tmpscore = generateScore(d,allCounties);
290             tmpscore = fdscore+fcscore-pdscore-pcscore;
291             // methodology: generate scores for all, sort, take the top
292             // ceil(1/50th) of points.
293             curfr[i]->setScore(tmpscore);
294             if(tmpscore >= hiscore){
295                 hiscore = tmpscore;
296                 best = curfr[i];
297             }
298             moveTract(curfr[i]->getTract(),BLANKDIST);
299         }
300         // sort descending scores here
301         sort(curfr.begin(),curfr.end(),compf);
302         //curfr.erase(unique(curfr.begin(),curfr.end(),eqf),curfr.end());
303         // do the movements;
304
305         j = (int)floor((double)curfr.size()/30.);
306         for(i=j;i != -1 ; i--){
307             moveTract(curfr[i]->getTract(),curfr[i]->getDistrict());
308         }
309         curfr.clear();
310         //moveTract(best->getTract(),best->getDistrict());
311         cout << "Score: " << generateScore(d,allCounties) << endl;
312     }
313
314     // District-by-District
315     /*
316     double pdoth,fdoth;
```

```
317         bool flag=false;
318         for(i=0; i < NDIST; i++){
319             curd = d[i];
320             flag = false;
321             do {
322                 j = randint(0,NTRACT-1);
323                 abba = allTracts[j];
324                 if(randdub() < (double)abba->getPop()/25000.)
325                     flag = true;
326             } while(abba->getDistrict() != BLANKDIST || !flag);
327             addme = abba;
328 
329             cout << "Starting District " << i+1 << endl;
330             /*
331             curd = d[i];
332             f = BLANKDIST->getTractList();
333             for(liter = f.begin(); liter != f.end(); liter++){
334                 moveTract(*liter,curd);
335                 tmpscore = generateScore(d,allCounties);
336                 if(tmpscore >= hiscore){
337                     hiscore = tmpscore;
338                     addme = *liter;
339                 }
340                 moveTract(*liter,BLANKDIST);
341             }*
342             moveTract(addme,curd);
343             done = false;
344             while(!done){
345                 curval = generateScore(d,allCounties);
346                 cout << "Score: " << curval << endl;
347                 hiscore = -50.;
348                 addme = NULL;
349                 f = curd->getFrontier();
350                 //cout << "Frontier has " << f.size() << " tracts" << endl;
351                 for(liter = f.begin(); liter != f.end(); liter++){
352                     // add liter to current allocation, getvalue, check and
353                     // unwind, settign hiscore and addme if necessary.
354                     checkme = (*liter)->getDistrict();
355                     if(checkme == curd){
356                         cerr << "There is a problem with frontier generation!"
357                             << endl;
358                     }
359                     if(checkme->isContiguous()){
360                         pdoth = checkme->getValue();
361                         pdscore = curd->getValue();
362                         iq =
363                             allCounties[CkeytoRkey[(*liter)->getCounty()]];
364                         pcscore = iq->getValue();
365                         moveTract(*liter,curd);
366                         fdoth = checkme->getValue();
367                         fdscore = curd->getValue();
368                         fcscore = iq->getValue();
369                         //tmpscore = generateScore(d,allCounties);
370                         tmpscore = fdscore+fcscore+fdoth - pcscore - pdscore
371                             - pdoth;
372                         if(tmpscore >= hiscore){
373                             addme = *liter;
374                             hiscore = tmpscore;
375                         }
376                         moveTract(*liter,checkme);
377                     }
378                 }
379                 if(addme == NULL){
380                     done = true;
381                 } else {
382                     moveTract(addme,curd);
383                 }
384             }
```

```
385         }
386     //*/
387     /*
388         for(i=0; i < NTRACT; i++){
389             allTracts[i]->setDistrict(d[0]);
390             d[0]->addToDistrict(allTracts[i]);
391         }
392         int seedind,k;
393         cout << "Beginning recursive initial districting" << endl;
394         for(i=1; i < NDIST; i++){
395             do {
396                 seedind = randint(0,NTRACT-1);
397             } while(allTracts[seedind]->getDistrict() != d[0]);
398 
399             // seed with self, neighbors, neighbors of neighbors
400             addneighrecur(allTracts[seedind],d[i],d[0],NLEVELS);
401             if((allTracts[seedind]->getN()).front()->getDistrict() !=
402                     allTracts[seedind]->getDistrict()){
403                 moveTract((allTracts[seedind]->getN()).front(),allTracts[seedind]->getDistrict()
                        );
404             }
405         }
406         // add District 0 possible elimination
407 
408         *
409         for(i=1; i < NDIST; i++){
410             if(thechosen[i]->getDistrict() != d[i]){
411                 moveTract(thechosen[i],d[i]);
412             }
413         }*
414 
415         for(i=0; i < NDIST; i++){
416             cout << "District " << i+1 << ": " << d[i]->getPop() << endl;
417             cout << " has " << (d[i]->getTractList()).size() << endl;
418         }
419 
420 
421         Allocation *a = new Allocation(d);
422         plotAllocation(a, "initial");
423         District **maybe;
424         District **curr = d;
425         for(i=0; i< 1000; i++){
426             //if(!(i%10))
427             clarify(allTracts);
428             cout << "Step " << i << " badness: " << getBadness(curr,distmat)
429                 << endl;
430             maybe = getNeighbor(curr,allTracts,distmat);
431 
432             if(!maybe){
433                 //cout << "I didn't improve!" << endl;
434             } else {
435                 curr = maybe;
436             }
437         }*/
438 
439         int sumpump=0;
440         for(i=0; i < NDIST+1; i++){
441             sumpump += d[i]->getPop();
442             cout << "District " << i+1 << ": " << d[i]->getPop() << endl;
443         }
444 
445         if(argc == 2){
446             list <Tract *>doolist;
447             list <Tract *>::iterator liter;
448             ofstream outfile(argv[1]);
449             double variance=0;
450             for(i=0; i < NDIST; i++){
451                 variance += pow((double)(d[i]->getPop()-AVGPEOPLE),2.);
```

```
452         }
453         variance = sqrt(variance);
454         outfile << variance << endl;
455         outfile << generateScore(d,allCounties) << endl;
456         for(i=0; i < NDIST; i++){
457             outfile << "D ";
458             doolist = d[i]->getTractList();
459             for(liter=doolist.begin(); liter != doolist.end(); liter++){
460                 outfile << (*liter)->getIndex() << " ";
461             }
462             outfile << endl;
463         }
464         outfile.close();
465     }
466 
467     cout << "Total population: " << sumpump << endl;
468     a = new Allocation(d);
469     plotAllocation(a, "testing");
470     return 0;
471 }
472 
473 // measures bc metric, returns max found...
474 double getBC(vector<Tract *> startingpoints,Tract *t){
475     int i;
476     double minv = 99999999999999999.;
477     double tmp;
478     for(i=0; i < startingpoints.size(); i++){
479         tmp = t->bcMetric(startingpoints[i]);
480         if(tmp < minv){
481             minv = tmp;
482         }
483     }
484 
485     return minv;
486 }
487 
488 vector <Fnode *> unionFrontier(District **d){
489     int i,j,k;
490     list<Tract *> f;
491     list<Tract *>::iterator liter,jiter,kiter;
492     Fnode *tmp;
493     bool flag;
494     vector <Fnode *> retval;
495     for(i=0; i < NDIST; i++){
496         f = d[i]->getFrontier();
497         /*
498         for(jiter = f.begin(); jiter != f.end(); jiter++){
499             flag = false;
500             for(kiter = jiter; kiter != f.end(); kiter++){
501                 if(((*jiter) == (*kiter)) && !flag){
502                     flag = true;
503                 } else if((*jiter) == (*kiter)){
504                     cout << "Duplicate in the frontier!" << endl;
505                 }
506             }
507         }
508         */
509         for(liter = f.begin(); liter != f.end(); liter++){
510             if((*liter)->getDistrict() == BLANKDIST){
511                 tmp = new Fnode(*liter,d[i]);
512                 retval.push_back(tmp);
513             }
514         }
515     }
516 
517     return retval;
518 }
519 
```

```
520 double generateScore(District **d, County **allCounties){
521     int i;
522     double pval=0;
523     double cval=0;
524 
525     for(i=0; i < NDIST; i++){
526         pval += d[i]->getValue();
527     }
528 
529     for(i=0; i < NCOUNTY; i++){
530         cval += allCounties[i]->getValue();
531     }
532 
533     if(PRINTHEU)
534         cout << "Population Score: " << pval << " County Score: " << cval << endl;
535     return pval+cval;
536 }
537 
538 void addneighrecur(Tract *t,District *changeto,District *background,int
539         levels){
540     if(levels == 0)
541         return;
542     if(t->getDistrict() == changeto || t->getDistrict() != background)
543         return;
544 
545         moveTract(t,changeto);
546         vector <Tract *> nvec;;
547         int j;
548         nvec = t->getN();
549         for(j=0; j < nvec.size(); j++){
550             addneighrecur(nvec[j],changeto,background,levels-1);
551         }
552 }
553 
554 void clarify(Tract **allTracts){
555     int i,j;
556     // if everything around me is another color, then I change
557     District *me,*oth;
558     vector <Tract *> n;
559     bool changeme;
560     for(i=0; i < NTRACT; i++){
561         me = allTracts[i]->getDistrict();
562         if(me->getTractList().size() <= 2){
563             continue;
564         }
565         n = allTracts[i]->getN();
566         if(n.size() > 0){
567             changeme = true;
568             for(j=0; j < n.size(); j++){
569                 if(me == n[j]->getDistrict()){
570                     changeme = false;
571                     break;
572                 }
573             }
574             /*
575             oth = n[0]->getDistrict();
576             if(oth != me){
577                 changeme = true;
578                 for(j=1; j < n.size(); j++){
579                     if(oth != n[j]->getDistrict()){
580                         changeme = false;
581                         break;
582                     }
583                 }
584             }*/
585             if(changeme){
586                 oth = n[randint(0,n.size()-1)]->getDistrict();
587                 cout << "Found enclave!" << endl;
```

```
588                     moveTract(allTracts[i],oth);
589                     changeme = false;
590                 }
591             }
592         }
593     }
594
595
596     District **getNeighbor(District **d, Tract** allTracts, double **distmat){
597         double curval = getBadness(d,distmat);
598         int i;
599
600         bool done = false;
601         Tract *tmp,*posc;
602         vector <Tract *> in;
603         while(!done){
604             tmp = allTracts[randint(0,NTRACT-1)];
605             if(!tmp->onPerimeter()){
606                 continue;
607             } else {
608                 in = tmp->getN();
609                 posc = in[randint(0,in.size()-1)];
610                 if(posc->getDistrict() != tmp->getDistrict())
611                     done = true;
612             }
613         }
614         /*
615         vector <Tract *> borders;
616
617         for(i=0;i < NTRACT; i++){
618             if(allTracts[i]->onPerimeter()){
619                 borders.push_back(allTracts[i]);
620             }
621         }
622
623         while(!done){
624             tmp = borders[randint(0,borders.size()-1)];
625             in = tmp->getN();
626             posc = in[randint(0,in.size()-1)];
627             if(posc->getDistrict() != tmp->getDistrict())
628                 done = true;
629         }
630         */
631         District *oldd = tmp->getDistrict();
632         District *newd = posc->getDistrict();
633         double movet;
634         double movec;
635         double swap;
636
637         // option one: let's move tmp to newd:
638
639         moveTract(tmp,newd);
640         movet = getBadness(d,distmat);
641         // huh. That didn't work. Let's try the other way...
642         moveTract(tmp,oldd);
643         moveTract(posc,oldd);
644         movec = getBadness(d,distmat);
645
646         // Try the swap...
647         moveTract(tmp,newd);
648         swap = getBadness(d,distmat);
649
650         list <double> l;
651         l.push_front(curval);
652         l.push_front(movet);
653         l.push_front(movec);
654         l.push_front(swap);
655
```

```
656     l.sort();
657     // current state: swapped
658     if(l.front() == curval){
659         moveTract(tmp,oldd);
660         moveTract(posc,newd);
661         return NULL;
662     } else if(l.front() == movet){
663         moveTract(posc,newd);
664         return d;
665     } else if(l.front() == movec){
666         moveTract(tmp,oldd);
667         return d;
668     } else {
669         return d;
670     }
671 }
672
673 // house cleaning to keep data structs in order
674 void moveTract(Tract *t, District *newd){
675     District *oldd = t->getDistrict();
676     if(oldd == newd){
677         cerr << "Trying to change to already fixed district!" << endl;
678         return;
679     }
680     list <Tract *> l = oldd->getTractList();
681     l.remove(t);
682     l = newd->getTractList();
683     l.push_front(t);
684     t->setDistrict(newd);
685     oldd->removeFromDistrict(t);
686     newd->addToDistrict(t);
687 }
688
689 double getBadness(District **d,double **distmat){
690     int i;
691     double sum=0;
692
693     // Linf norm (max)
694     /*
695     for(i=0; i < NDIST; i++){
696         if(d[i]->getPop() > sum){
697             sum = d[i]->getPop();
698         }
699     }*/
700     // L2 norm (variance):
701
702     for(i=0; i < NDIST; i++){
703         sum += pow(d[i]->getPop()-AVGPEOPLE,2);
704     }
705     sum = sqrt(sum); // add constant factor here at some point
706
707
708     double dist=0;
709     list <Tract *>lind;
710     list <Tract *>::iterator iti;
711     list <Tract *>::iterator jtj;
712     double mydist=0;
713     for(i=0; i < NDIST; i++){
714         lind = d[i]->getTractList();
715         for(iti = lind.begin(); iti != lind.end(); iti++){
716             for(jtj = iti; jtj != lind.end(); jtj++){
717                 mydist += distmat[(*iti)->getIndex()][(*jtj)->getIndex()];
718             }
719         }
720         dist +=
721             mydist/(lind.size()*(lind.size()-1)*sqrt(d[i]->getArea()));
722         mydist = 0;
723     }
```

```
724
725     dist = dist * 700000;
726     cout << "Sum of Distances Metric: " << dist << " Population Metric: " << sum << endl;
727     return dist+sum;
728 }
729
730
731 void plotAllocation(Allocation *a,string fname){
732     // plots an Allocation to a file
733
734     const char *pszDriverName = "ESRI Shapefile";
735     OGRSFDriver *poDriver;
736
737     OGRRegisterAll();
738
739     poDriver =
740         OGRSFDriverRegistrar::GetRegistrar()->GetDriverByName(
741                 pszDriverName);
742     if(!poDriver){
743         cerr << "Could not initialize driver for writing!" << endl;
744         return;
745     }
746
747     OGRDataSource *poDS;
748     OGRLayer *layer;
749     District **d = a->getDistricts();
750     int i;
751     string curname,lname;
752     OGRFeature *tmpf;
753     list <Tract *>tracts;
754     list <Tract *>::iterator iter;
755
756     for(i=0; i < NDIST; i++){
757         tracts = d[i]->getTractList();
758         curname = fname + inttostring(i) + ".shp";
759         poDS = poDriver->CreateDataSource(fname.c_str(), NULL);
760         if(!poDS){
761             cerr << "Could not create output file!" << endl;
762             return;
763         }
764         lname = "District " + inttostring(i+1);
765         layer = poDS->CreateLayer(lname.c_str(), NULL, wkbUnknown,NULL);
766         if(!layer){
767             cerr << "Layer creation failed!" << endl;
768             return;
769         }
770
771         for(iter = tracts.begin(); iter != tracts.end(); iter++){
772             tmpf = new OGRFeature(layer->GetLayerDefn());
773             tmpf->SetGeometry((*iter)->getGeo());
774             if(layer->CreateFeature(tmpf) != OGRERR_NONE){
775                 cerr << "Could not create feature!" << endl;
776                 return;
777             }
778             OGRFeature::DestroyFeature(tmpf);
779         }
780         OGRDataSource::DestroyDataSource(poDS);
781     }
782 }
```

```
1 #include "ogrsf_frmts.h"
2 #include <iostream>
3 #include <fstream>
4 #include <iomanip>
5 #include <string>
6 #include <map>
7 #include "Tract.h"
8 //#include "County.h"
9 //#include "District.h"
```

```
10 #include "Allocation.h"
11 //#include "rng.h"
12 #include <sstream>
13 #include <cstdlib>
14 #include <ctime>
15 #include <vector>
16 #include "Fnode.h"
17 #include <algorithm>
18
19 const int NTRACT = 4907;
20 const int NDIST = 29;
21 const double AVGPEOPLE = 18976457./(float)NDIST;
22 const int NCOUNTY = 62;
23 //const int NLEVELS = 20;
24 District *BLANKDIST;
25 const bool PRINTHEU = false;
26
27 using namespace std;
28
29 void plotAllocation(Allocation *a,string fname);
30 void moveTract(Tract *t, District *newd);
31 vector <Fnode *> unionFrontier(District **d);
32 double getBC(vector<Tract *> startingpoints,Tract *t);
33 District *largestD(District **d);
34 double partTwoScore(District **d,County **allCounties);
35 District *smallestD(District **d);
36 vector <Fnode *> addingMoves(District *dis);
37 vector <Fnode *> reducingMoves(District *dis);
38 District *nextD(District **d);
39
40 bool compf(Fnode *lhs, Fnode *rhs){
41     // greater than, not less than, b/c we want to sort descending
42     return lhs->getScore() > rhs->getScore();
43 }
44
45 bool eqf(Fnode *lhs, Fnode *rhs){
46     return lhs->getScore() == rhs->getScore();
47 }
48
49 bool eq_func(Tract *lhs, Tract *rhs){
50     return lhs == rhs;
51 }
52 bool compbefore(Fnode *lhs, Fnode *rhs){
53     if(rhs->getTract() >= lhs->getTract()){
54         return true;
55     } else if(rhs->getTract() == lhs->getTract()){
56         if(rhs->getDistrict() >= lhs->getDistrict()){
57             return true;
58         }
59     }
60     return false;
61 }
62
63 bool eqbefore(Fnode *lhs, Fnode *rhs){
64     return((rhs->getTract() == lhs->getTract()) &&
65             (rhs->getDistrict() == lhs->getDistrict()));
66 }
67
68 bool comp_func(Tract *lhs, Tract *rhs){
69     return lhs < rhs;
70 }
71
72 string inttostring(const int i){
73     ostringstream stream;
74     stream << i;
75     return stream.str();
76 }
77
```

```
78  double randdub(){
79      return rand()/(double)RAND_MAX;
80  }
81  //returns between lo and hi inclusive
82  int randint(int low, int high){
83      return(low+(int)floor(randdub()*(high-low+1)));
84  }
85
86  vector <Tract *>copyvec(const vector<Tract *> &in){
87      int i;
88      vector <Tract *> returnval;
89      for(i=0;i<in.size();i++){
90          returnval[i] = in[i];
91      }
92  }
93
94  int idFromString(char *s,map<string,int> m){
95      string k(s);
96
97      return m[k];
98  }
99
100 int main(int argc, char * const argv[]) {
101     srand((unsigned)time(NULL));
102
103     OGRRegisterAll();
104
105     OGRDataSource *myfile;
106
107     myfile = OGRSFDriverRegistrar::Open("./polygons/", FALSE);
108     if(myfile == NULL){
109         cerr << "Can't open file" << endl;
110         return 1;
111     }
112     cout << "Opened file appropriately!" << endl;
113     cout << "File has " << myfile->GetLayerCount() << " layers" << endl;
114
115     OGRLayer *layer = myfile->GetLayer(0);
116     if(!layer){
117         cerr << "Cannot open layer" << endl;
118         return 1;
119     }
120
121     cout << "Layer has " << layer->GetFeatureCount() << " features" <<
122         endl;
123     int numtracts = layer->GetFeatureCount();
124     int i,j;
125     OGRFeature *feat;
126     int populationindex;
127     int totalpop = 0;
128     map<string,int> IDtoIref;
129     map<int,int> CkeytoRkey; // county key in file to our real keys.
130     Tract *allTracts[NTRACT];
131     bool **bmat = new bool*[NTRACT];
132     double **distmat = new double*[NTRACT];
133
134     Allocation *a;
135     County **allCounties = new County*[NCOUNTY];
136     for(i=0; i < NCOUNTY; i++){
137         allCounties[i] = new County();
138     }
139
140     int cindex=-1;
141     for(i=0; i < numtracts; i++){
142         feat = layer->GetNextFeature();
143         if(!feat){
144             cerr << "Could not read feature, exiting!" << endl;
145             return 1;
```

```
146             }
147             allTracts[i] = new Tract(feat,i);
148             IDtoIref[allTracts[i]->getID()] = i;
149             // Link to counties...
150             if(CkeytoRkey.count(allTracts[i]->getCounty()) == 0){
151                 cindex++;
152                 CkeytoRkey[allTracts[i]->getCounty()] = cindex;
153             }
154             allCounties[CkeytoRkey[allTracts[i]->getCounty()]]->addToCounty(allTracts[i]);
155             allTracts[i]->setCounty(allCounties[CkeytoRkey[allTracts[i]->getCounty()]]);
156 
157             delete feat;
158             feat = NULL;
159         }
160 
161         cout << "beginning to read border file..." << endl;
162         ifstream bo;
163         bo.open("border.txt");
164         for(i=0; i < NTRACT; i++){
165             bmat[i] = new bool[NTRACT];
166             for(j=0; j < NTRACT; j++){
167                 bo >> bmat[i][j];
168             }
169         }
170         bo.close();
171         cout << "finished reading border file" << endl;
172 
173       /*
174         cout << "beginning calculating centroid distances" << endl;
175         for(i=0; i < NTRACT; i++){
176             distmat[i] = new double[NTRACT];
177             for(j=0; j < NTRACT; j++){
178                 if(j < i){
179                     distmat[i][j] = distmat[j][i];
180                 } else {
181                     distmat[i][j] = allTracts[i]->distC(allTracts[j]);
182                 }
183             }
184         }
185         cout << "finished calculating centroid distances" << endl;
186     */
187         District *d[NDIST+1]; // d[NDIST] = blank canvas....
188         for(i=0; i < NDIST+1; i++){
189             d[i] = new District();
190         }
191 
192         BLANKDIST = d[NDIST];
193         // Read in file here....
194         cout << "opening input file...." << endl;
195         if(argc >= 2){
196             list <Tract *>doolist;
197             list <Tract *>::iterator liter;
198             ifstream infile(argv[1]);
199             if(!infile){
200                     cerr << "Could not open " << argv[1] << endl;
201                     return 1;
202             }
203             double upp;
204             infile >> upp;
205             cout << "Variance: " << upp << endl;
206             infile >> upp;
207             cout << "Score: " << upp << endl;
208             int inp;
209             for(i=-1; (i < NDIST) && !infile.eof(); i++){
210                 infile >> inp;
211                 while((inp != -1) && !infile.eof()){
212                     allTracts[inp]->setDistrict(d[i]);
213                     d[i]->addToDistrict(allTracts[inp]);
```

```
214                     infile >> inp;
215                 }
216             }
217
218             infile.close();
219         } else {
220                 cerr << "Must call an input file..." << endl;
221                 return 1;
222         }
223
224
225         double** intermat = new double*[NTRACT];
226         double *myp = new double[NTRACT];
227         OGRGeometry *ia;
228         OGRGeometry *ib;
229         OGRGeometry *u;
230         double sz;
231         for(i=0; i < NTRACT; i++){
232             ia = allTracts[i]->getGeo();
233             myp[i] = (((OGRPolygon *)ia)->getExteriorRing())->get_Length();
234         }
235
236         /*
237         for(i=0; i < NTRACT; i++){
238             intermat[i] = new double[NTRACT];
239             for(j=0; j < NTRACT; j++){
240                 if(!bmat[i][j]){
241                     intermat[i][j]=0;
242                     continue;
243                 }
244
245                 if(i > j){
246                     intermat[i][j] = intermat[j][i];
247                     continue;
248                 }
249                 ia = allTracts[i]->getGeo();
250                 ib = allTracts[j]->getGeo();
251                 u = ia->Union(ib);
252                 sz = (((OGRPolygon *)u)->getExteriorRing())->get_Length();
253                 intermat[i][j] = (double)(myp[i]+myp[j]-sz)/(double)2.;
254                 if(intermat[i][j] < 0){
255                     cout << "Negative for " << allTracts[i]->getID() <<
256                         " and " << allTracts[j]->getID() << endl;
257                     intermat[i][j] = max(myp[i],myp[j]);
258                 } else if(intermat[i][j] < 1e-5){
259                     intermat[i][j] = 0; //set to 0 so that they don't border
260                 }
261                 //cout << intermat[i][j] << endl;
262             }
263         }*/
264         cout << "Done processing unions" << endl;
265     /*
266         int sm = IDtoIref["1928646"];
267         int top = IDtoIref["1928680"];
268         int left = IDtoIref["1928388"];
269         int rt = IDtoIref["1928582"];
270
271         cout << myp[sm] << " " << intermat[sm][top] << " " <<
272             intermat[sm][left] << " " << intermat[sm][rt] << endl;
273         cout << myp[sm] << " " << myp[top] << " " << myp[left] << " "
274             << myp[rt] << endl;
275
276         for(i=0; i < NTRACT; i++){
277             for(j=0; j < NTRACT; j++){
278                 if(bmat[i][j] >){
279                     allTracts[i]->addPerim(allTracts[j],intermat[i][j]);
280                 }
281             }
```

```
282         }
283     */
284
285         vector <Tract *> n;
286         for(i=0; i < NTRACT; i++){
287             for(j=0; j < NTRACT; j++){
288                 if(bmat[i][j]){
289                     n.push_back(allTracts[j]);
290                 }
291             }
292             allTracts[i]->setN(n);
293             n.clear();
294         }
295         /*
296         cout << myp[IDtoIref["754210"]] << endl;
297         cout << myp[IDtoIref["759105"]] << endl;
298         cout << intermat[IDtoIref["754210"]][IDtoIref["759105"]] << endl;
299         cout << intermat[IDtoIref["578438"]][IDtoIref["593495"]] << endl;
300         */
301         //     " and " << calp->get_Length() << endl;
302         //for(i=0; i < NDIST; i++){
303         //     d[i]->getIsoPerim();
304         //}
305         District *dsm; //smallest district;
306         District *dlg; // largest district;
307         District *you,*me;
308         vector<Fnode *> addingf;
309         //Fnode *bestadd;
310         County *iq;
311         double pcscore,fcscore;
312         double pcompactyou,pcompactme;
313         double fcompactyou,fcompactme;
314         double pvaryou,pvarme;
315         double fvaryou,fvarme;
316         double bestscore=-1e300;
317         double tmpscore;
318         // we do not need to consider my past compactness or my past
319         // variance because all possible moves will consider that. Ignore.
320         double varscore = 0;
321         double pscore = -1e347;
322         double curscore = -1e300;
323         District *nextd;
324         vector<Fnode *> adds;
325         vector<Fnode *> removes;
326         Fnode *bestadd;
327         Fnode *bestremove;
328         District *youtakeme;
329         District *itakeyou;
330         vector<District *> myborders;
331         vector<Tract *> swappage;
332         District *block;
333         double prevscore,futscore;
334         int count;
335         for(count = 0; count < 500; count++){
336             cout << "Iteration " << count+1 << endl;
337             //do{
338             pscore = curscore;
339             // add to smallest District...
340             bestadd = NULL;
341             bestremove = NULL;
342             bestscore = -1e300;
343
344             nextd = nextD(d);
345             adds = addingMoves(nextd);
346             removes = reducingMoves(nextd);
347             me = nextd;
348
349             /*
```

```
350             if(count < 200){
351             myborders = me->whatBordersMe();
352             block = myborders[randint(0,myborders.size()-1)];
353             swappage = me->sharesBorder(block);
354             // swap out, then swap in....
355             prevscore = me->score() + block->score();
356             for(i=0; i < swappage.size(); i++){
357                 moveTract(swappage[i],me);
358             }
359             futscore = me->score() + block->score();
360             for(i=0; i < swappage.size(); i++){
361                 moveTract(swappage[i],block);
362             }
363             if((me->isContiguous() && block->isContiguous())){
364                 tmpscore = futscore - prevscore;
365                 if(tmpscore > 0){
366                     for(i=0; i < swappage.size(); i++){
367                         moveTract(swappage[i],me);
368                     }
369                     cout << "Made massive swap!" << endl;
370                     continue;
371                 }
372             }
373             swappage = block->sharesBorder(block);
374             for(i=0; i < swappage.size(); i++){
375                 moveTract(swappage[i],block);
376             }
377             futscore = me->score() + block->score();
378             for(i=0; i < swappage.size(); i++){
379                 moveTract(swappage[i],me);
380             }
381             if((me->isContiguous() && block->isContiguous())){
382                 tmpscore = futscore - prevscore;
383                 if(tmpscore > 0){
384                     for(i=0; i < swappage.size(); i++){
385                         moveTract(swappage[i],block);
386                     }
387                     cout << "Made massive swap!" << endl;
388                     continue;
389                 }
390             }
391             }*/
392             // consider all adds
393             for(i=0; i < adds.size(); i++){
394                 itakeyou = (adds[i]->getTract())->getDistrict();
395                 prevscore = itakeyou->score() + me->score();
396                 moveTract(adds[i]->getTract(),me);
397                 if(!itakeyou->isContiguous()){
398                     moveTract(adds[i]->getTract(),itakeyou);
399                     continue;
400                 }
401                 futscore = itakeyou->score() + me->score();
402                 tmpscore = futscore - prevscore;
403                 if(tmpscore > bestscore){
404                     bestscore = tmpscore;
405                     bestadd = adds[i];
406                     bestremove = NULL;
407                 }
408                 moveTract(adds[i]->getTract(),itakeyou);
409             }
410
411             // consider all removes
412             for(i=0; i < removes.size(); i++){
413                 youtakeme = removes[i]->getDistrict();
414                 prevscore = youtakeme->score() + me->score();
415                 moveTract(removes[i]->getTract(),youtakeme);
416                 if(!me->isContiguous()){
417                     moveTract(removes[i]->getTract(),me);
```

```
418                 continue;
419             }
420             futscore = me->score() + youtakeme->score();
421             tmpscore = futscore - prevscore;
422             if(tmpscore > bestscore){
423                 bestscore = tmpscore;
424                 bestadd = NULL;
425                 bestremove = removes[i];
426             }
427             moveTract(removes[i]->getTract(),me);
428         }
429 
430         // consider all swaps
431         if(/*bestscore < 0 && randdub() < 0.9*/ true){
432             for(i=0; i < removes.size(); i++){
433                 youtakeme =removes[i]->getDistrict();
434                 for(j=0; j < adds.size(); j++){
435                     itakeyou = (adds[j]->getTract())->getDistrict();
436                     if(youtakeme == itakeyou){
437                         prevscore = youtakeme->score() + me->score();
438                     } else {
439                         prevscore = youtakeme->score() + me->score() +
440                             itakeyou->score();
441                     }
442                     moveTract(removes[i]->getTract(),youtakeme);
443                     moveTract(adds[j]->getTract(),me);
444                     if(!itakeyou->isContiguous() ||
445                             !me->isContiguous()){
446                         moveTract(removes[i]->getTract(),me);
447                         moveTract(adds[j]->getTract(),itakeyou);
448                         continue;
449                     }
450                     if(youtakeme != itakeyou){
451                         futscore = me->score() + youtakeme->score() +
452                             itakeyou->score();
453                     } else {
454                         futscore = me->score() + youtakeme->score();
455                     }
456                     tmpscore = futscore - prevscore;
457                     if(tmpscore > bestscore){
458                         bestscore = tmpscore;
459                         bestadd = adds[j];
460                         bestremove = removes[i];
461                     }
462                     moveTract(removes[i]->getTract(),me);
463                     moveTract(adds[j]->getTract(),itakeyou);
464                 }
465             }
466         }
467         if(bestscore > 0){
468             // make the moves, clear the stuff
469             if(bestadd){
470                 moveTract(bestadd->getTract(),me);
471             }
472             if(bestremove){
473                 moveTract(bestremove->getTract(),bestremove->getDistrict());
474             }
475             if(bestadd && bestremove){
476                 cout << "Swap is the best move!" << endl;
477             }
478         }
479         adds.clear();
480         removes.clear();
481         curscore = partTwoScore(d,allCounties);
482         cout << "Current Score: " << curscore << endl;
483         //} while(bestadd || bestremove);
484         varscore = 0;
485         for(i=0; i < NDIST; i++){
```

```
486             varscore += d[i]->varScore();
487         }
488         //} while(varscore < -1);
489 }
490
491 int sumpump=0;
492 for(i=0; i < NDIST; i++){
493     sumpump += d[i]->getPop();
494     cout << "District " << i+1 << ": " << d[i]->getPop() << endl;
495 }
496
497 if(argv[2]){
498 ofstream ogil(argv[2]);
499 ogil << varscore << endl;
500 ogil << partTwoScore(d,allCounties) << endl;
501 list<Tract *> lst;
502 list<Tract *>::iterator liter;
503 for(i=0; i < NDIST; i++){
504     ogil << "-1 ";
505     lst = d[i]->getTractList();
506     for(liter = lst.begin(); liter != lst.end(); liter++){
507         ogil << (*liter)->getIndex() << " ";
508     }
509     ogil << endl;
510 }
511
512 ogil.close();
513 }
514 cout << "Total population: " << sumpump << endl;
515 a = new Allocation(d);
516 plotAllocation(a, "parttwo_finish");
517 return 0;
518 }
519
520 //bool randnext = false;
521 District *nextD(District **d){
522
523     District *smallest = d[randint(0,NDIST-1)];
524     /*
525     if(randnext){
526         smallest = d[randint(0,NDIST-1)];
527         randnext = false;
528
529     } else {
530         int i;
531         smallest = d[0];
532         double score = d[0]->score();
533         double ts;
534         for(i=1; i < NDIST; i++){
535             ts = d[i]->score();
536             if(ts < score){
537                 smallest = d[i];
538                 score = ts;
539             }
540         }
541     randnext = true;
542     }*/
543     return smallest;
544 }
545
546
547 District *smallestD(District **d){
548     int i;
549     District *smallest = d[0];
550     int smpop = d[0]->getPop();
551     for(i=1; i < NDIST; i++){
552         if(d[i]->getPop() < smpop){
553             smpop = d[i]->getPop();
```

```
554                 smallest = d[i];
555             }
556         }
557         return smallest;
558     }
559
560     District *largestD(District **d){
561         int i;
562         District *largest = d[0];
563         int smpop = d[0]->getPop();
564         for(i=1; i < NDIST; i++){
565             if(d[i]->getPop() > smpop){
566                 smpop = d[i]->getPop();
567                 largest = d[i];
568             }
569         }
570         return largest;
571     }
572
573     double partTwoScore(District **d,County **allCounties){
574         int i;
575         double compact=0,var=0,county=0,ncscore=0;
576         for(i=0; i < NDIST; i++){
577             compact += d[i]->compactScore();
578             var += d[i]->varScore();
579             county += d[i]->countyScore();
580             ncscore += d[i]->newcountyScore();
581         }
582
583         /*
584         for(i=0; i < NCOUNTY; i++){
585             county += allCounties[i]->getValue();
586         }*/
587         cout << "Variance: " << var << " Compactness: " << compact <<
588             " County: " << county << " New County Score: " << ncscore << endl;
589
590         return var + compact + county + ncscore;
591     }
592
593     vector <Fnode *> reducingMoves(District *dis){
594         list <Tract *> f = dis->getPerimeter();
595         list <Tract *>::iterator liter;
596         Fnode *tmp;
597         vector <Fnode *> retval;
598         int i;
599         vector <District *> otherD;
600
601         for(liter = f.begin(); liter != f.end(); liter++){
602             otherD = (*liter)->getNColors();
603             for(i=0; i < otherD.size(); i++){
604                 tmp = new Fnode(*liter,otherD[i]);
605                 retval.push_back(tmp);
606             }
607         }
608
609         return retval;
610     }
611
612     vector <Fnode *> addingMoves(District *dis){
613         list <Tract *> f = dis->getFrontier();
614         list <Tract *>::iterator liter;
615         Fnode *tmp;
616         vector <Fnode *> retval;
617
618         for(liter = f.begin(); liter != f.end(); liter++){
619             tmp = new Fnode(*liter,dis);
620             retval.push_back(tmp);
621         }
```

```
622     if(f.size() == 0){
623         cout << "blank frontier" << endl;
624     }
625     if(retval.size() == 0){
626         cout << "blank retval" << endl;
627     }
628 
629     return retval;
630 }
631 
632 vector <Fnode *> unionFrontier(District **d){
633     int i;
634     list<Tract *> f;
635     list<Tract *>::iterator liter;
636     Fnode *tmp;
637     bool flag;
638     vector <Fnode *> retval;
639     for(i=0; i < NDIST; i++){
640         f = d[i]->getFrontier();
641         for(liter = f.begin(); liter != f.end(); liter++){
642             if((*liter)->getDistrict() == BLANKDIST){
643                 tmp = new Fnode(*liter,d[i]);
644                 retval.push_back(tmp);
645             }
646         }
647     }
648 
649     return retval;
650 }
651 
652 // house cleaning to keep data structs in order
653 void moveTract(Tract *t, District *newd){
654     District *oldd = t->getDistrict();
655     if(oldd == newd){
656         cerr << "Trying to change to already fixed district!" << endl;
657         return;
658     }
659     list <Tract *> l = oldd->getTractList();
660     l.remove(t);
661     l = newd->getTractList();
662     l.push_front(t);
663     t->setDistrict(newd);
664     oldd->removeFromDistrict(t);
665     newd->addToDistrict(t);
666 }
667 
668 void plotAllocation(Allocation *a,string fname){
669     // plots an Allocation to a file
670 
671     const char *pszDriverName = "ESRI Shapefile";
672     OGRSFDriver *poDriver;
673 
674     OGRRegisterAll();
675 
676     poDriver =
677         OGRSFDriverRegistrar::GetRegistrar()->GetDriverByName(
678                 pszDriverName);
679     if(!poDriver){
680         cerr << "Could not initialize driver for writing!" << endl;
681         return;
682     }
683 
684     OGRDataSource *poDS;
685     OGRLayer *layer;
686     District **d = a->getDistricts();
687     int i;
688     string curname,lname;
689     OGRFeature *tmpf;
```

```
690     list <Tract *>tracts;
691     list <Tract *>::iterator iter;
692 
693     for(i=0; i < NDIST; i++){
694         tracts = d[i]->getTractList();
695         curname = fname + inttostring(i) + ".shp";
696         poDS = poDriver->CreateDataSource(fname.c_str(), NULL);
697         if(!poDS){
698             cerr << "Could not create output file!" << endl;
699             return;
700         }
701         lname = "District_" + inttostring(i+1);
702         layer = poDS->CreateLayer(lname.c_str(), NULL, wkbUnknown,NULL);
703         if(!layer){
704             cerr << "Layer creation failed!" << endl;
705             return;
706         }
707 
708         for(iter = tracts.begin(); iter != tracts.end(); iter++){
709             tmpf = new OGRFeature(layer->GetLayerDefn());
710             tmpf->SetGeometry((*iter)->getGeo());
711             if(layer->CreateFeature(tmpf) != OGRERR_NONE){
712                 cerr << "Could not create feature!" << endl;
713                 return;
714             }
715             OGRFeature::DestroyFeature(tmpf);
716         }
717         OGRDataSource::DestroyDataSource(poDS);
718     }
719 }
```

```
 1 #include "ogrsf_frmts.h"
 2 #include <iostream>
 3 #include <fstream>
 4 #include <iomanip>
 5 #include <string>
 6 #include <map>
 7 #include "Tract.h"
 8 //#include "County.h"
 9 #include "District.h"
10 #include "Allocation.h"
11 //#include "rng.h"
12 #include <sstream>
13 #include <cstdlib>
14 #include <ctime>
15 #include <vector>
16 #include "Fnode.h"
17 #include <algorithm>
18 
19 const int NTRACT = 4907;
20 const int NDIST = 29;
21 const double AVGPEOPLE = 18976457./(float)NDIST;
22 const int NCOUNTY = 62;
23 //const int NLEVELS = 20;
24 District *BLANKDIST;
25 const bool PRINTHEU = false;
26 
27 using namespace std;
28 
29 void plotAllocation(Allocation *a,string fname);
30 
31 string inttostring(const int i){
32     ostringstream stream;
33     stream << i;
34     return stream.str();
35 }
36 
37 int main(int argc, char *argv[]){
38     srand((unsigned)time(NULL));
```

```
39
40      OGRRegisterAll();
41
42      OGRDataSource *myfile;
43
44      myfile = OGRSFDriverRegistrar::Open("./polygons/", FALSE);
45      if(myfile == NULL){
46          cerr << "Can't open file" << endl;
47          return 1;
48      }
49      cout << "Opened file appropriately!" << endl;
50      cout << "File has " << myfile->GetLayerCount() << " layers" << endl;
51
52      OGRLayer *layer = myfile->GetLayer(0);
53      if(!layer){
54          cerr << "Cannot open layer" << endl;
55          return 1;
56      }
57
58      cout << "Layer has " << layer->GetFeatureCount() << " features" <<
59          endl;
60      int numtracts = layer->GetFeatureCount();
61      int i,j;
62      OGRFeature *feat;
63      int populationindex;
64      int totalpop = 0;
65      map<string,int> IDtoIref;
66      map<int,int> CkeytoRkey; // county key in file to our real keys.
67      Tract *allTracts[NTRACT];
68      bool **bmat = new bool*[NTRACT];
69      double **distmat = new double*[NTRACT];
70
71      Allocation *a;
72
73      County **allCounties = new County*[NCOUNTY];
74      for(i=0; i < NCOUNTY; i++){
75          allCounties[i] = new County();
76      }
77
78      int cindex=-1;
79      for(i=0; i < numtracts; i++){
80          feat = layer->GetNextFeature();
81          if(!feat){
82              cerr << "Could not read feature, exiting!" << endl;
83              return 1;
84          }
85          allTracts[i] = new Tract(feat,i);
86          IDtoIref[allTracts[i]->getID()] = i;
87          // Link to counties...
88          if(CkeytoRkey.count(allTracts[i]->getCounty()) == 0){
89              cindex++;
90              CkeytoRkey[allTracts[i]->getCounty()] = cindex;
91          }
92          allCounties[CkeytoRkey[allTracts[i]->getCounty()]]->addToCounty(allTracts[i]);
93          allTracts[i]->setCounty(allCounties[CkeytoRkey[allTracts[i]->getCounty()]]);
94
95          delete feat;
96          feat = NULL;
97      }
98
99      cout << "beginning to read border file..." << endl;
100     ifstream bo;
101     bo.open("border.txt");
102     for(i=0; i < NTRACT; i++){
103         bmat[i] = new bool[NTRACT];
104         for(j=0; j < NTRACT; j++){
105             bo >> bmat[i][j];
106         }
```

```
107       }
108       bo.close();
109       cout << "finished reading border file" << endl;
110      District *d[NDIST+1]; // d[NDIST] = blank canvas....
111      for(i=0; i < NDIST+1; i++){
112              d[i] = new District();
113      }
114
115       vector <Tract *> n;
116       for(i=0; i < NTRACT; i++){
117           for(j=0; j < NTRACT; j++){
118               if(bmat[i][j]){
119                   n.push_back(allTracts[j]);
120               }
121           }
122           allTracts[i]->setN(n);
123           n.clear();
124       }
125
126       BLANKDIST = d[NDIST];
127       // Read in file here....
128       cout << "opening input file...." << endl;
129       if(argc >= 2){
130           list <Tract *>doolist;
131           list <Tract *>::iterator liter;
132           ifstream infile(argv[1]);
133           if(!infile){
134                   cerr << "Could not open " << argv[1] << endl;
135                   return 1;
136           }
137           double upp;
138           infile >> upp;
139           cout << "Variance: " << upp << endl;
140           infile >> upp;
141           cout << "Score: " << upp << endl;
142           int inp;
143           for(i=-1; (i < NDIST) && !infile.eof(); i++){
144               infile >> inp;
145               while((inp != -1) && !infile.eof()){
146                   allTracts[inp]->setDistrict(d[i]);
147                   d[i]->addToDistrict(allTracts[inp]);
148                   infile >> inp;
149               }
150           }
151
152           infile.close();
153       } else {
154               cerr << "Must call an input file..." << endl;
155               return 1;
156       }
157
158       for(i=0; i < NCOUNTY; i++){
159               allCounties[i]->printCounty();
160       }
161       double varScore = d[0]->varScore();
162       double cScore = d[0]->countyScore();
163       double compact = d[0]->compactScore();
164       double minpop = d[0]->getPop();
165       double maxpop = d[0]->getPop();
166       cout << "District " << 1 << " " << d[0]->getPop() << endl;
167       for(i=1; i < NDIST; i++){
168               //cScore += d[i]->countyScore();
169               //compact += d[i]->compactScore();
170               varScore += d[i]->varScore();
171               if(d[i]->getPop() > maxpop){
172                       maxpop = d[i]->getPop();
173               }
174               if(d[i]->getPop() < minpop){
```

更多学习资料，请关注淘宝店铺：学神资料站　https://shop156050543.taobao.com/

```
175                         minpop = d[i]->getPop();
176                 }
177                 cout << "District " << i+1 << " " << d[i]->getPop() << endl;
178         }
179         cout << "Variance: " << varScore << " Max: " << maxpop << " Min: "
180                 << minpop << endl;
181         cout << "County: " << cScore << " Compact: " << compact << endl;
182 
183         a = new Allocation(d);
184         plotAllocation(a,argv[2]);
185         return 0;
186        }
187 
188 void plotAllocation(Allocation *a,string fname){
189         // plots an Allocation to a file
190 
191         const char *pszDriverName = "ESRI Shapefile";
192         OGRSFDriver *poDriver;
193 
194         OGRRegisterAll();
195 
196         poDriver =
197                 OGRSFDriverRegistrar::GetRegistrar()->GetDriverByName(
198                         pszDriverName);
199         if(!poDriver){
200                 cerr << "Could not initialize driver for writing!" << endl;
201                 return;
202         }
203 
204         OGRDataSource *poDS;
205         OGRLayer *layer;
206         District **d = a->getDistricts();
207         int i;
208         string curname,lname;
209         OGRFeature *tmpf;
210         list <Tract *>tracts;
211         list <Tract *>::iterator iter;
212 
213         for(i=0; i < NDIST; i++){
214                 tracts = d[i]->getTractList();
215                 curname = fname + inttostring(i) + ".shp";
216                 poDS = poDriver->CreateDataSource(fname.c_str(), NULL);
217                 if(!poDS){
218                         cerr << "Could not create output file!" << endl;
219                         return;
220                 }
221                 lname = "District " + inttostring(i+1);
222                 layer = poDS->CreateLayer(lname.c_str(), NULL, wkbUnknown,NULL);
223                 if(!layer){
224                         cerr << "Layer creation failed!" << endl;
225                         return;
226                 }
227 
228                 for(iter = tracts.begin(); iter != tracts.end(); iter++){
229                         tmpf = new OGRFeature(layer->GetLayerDefn());
230                         tmpf->SetGeometry((*iter)->getGeo());
231                         if(layer->CreateFeature(tmpf) != OGRERR_NONE){
232                                 cerr << "Could not create feature!" << endl;
233                                 return;
234                         }
235                         OGRFeature::DestroyFeature(tmpf);
236                 }
237                 OGRDataSource::DestroyDataSource(poDS);
238         }
239 }
```